AUDITABLE ETL PIPELINES IN REGULATED ENVIRONMENTS USING A DEVSECOPS-BASED TOOLCHAIN

Daniel Dickerson

Bachelor of Engineering (Honours)
Software Engineering

School of Engineering
Macquarie University

November 2025

Supervisor: Dr Ansgar Fehnker

ACKNOWLEDGMENTS

I would like to thank my supervisor, Dr Ansgar Fehnker, for his guidance and feedback throughout this project; Dr Kate Stefanov, for her ongoing advice and coordination through the weekly thesis sessions; and my industry co-supervisors, Aneesh Kondepuri and Richard Hawkes, for providing feedback and assistance throughout this project.

STATEMENT OF CANDIDATE

I, Daniel Dickerson, declare that this report, submitted as part of the requirement for the award of Bachelor of Engineering in the School of Engineering, Macquarie University, is entirely my own work unless otherwise referenced or acknowledged. This document has not been submitted for qualification or assessment at any other academic institution.

Student's Name: Daniel Dickerson

Student's Signature: Daniel Dickerson

Date: June 3, 2026

ABSTRACT

This thesis explores how a DevSecOps toolchain can be designed to support ETL (Extract–Transform–Load) workflows in regulated industries, with a focus on auditability and governance. It applies a Design Science Research approach to design a control framework that translates regulatory expectations—such as those in APRA's CPS 230, 234, and 510—into technical assurance mechanisms.

The work to date defines a conceptual baseline pipeline using Apache Airflow for orchestration, PySpark on EMR for transformation, and Amazon S3 for storage. Infrastructure is provisioned through Terraform and deployed via GitHub Actions, forming a reproducible but ungoverned model that establishes how data moves, where control resides, and what evidence is naturally produced. Analysis of this baseline identifies gaps in governance, integrity verification, and end-to-end lineage, forming the foundation for the DevSecOps architecture that follows.

The study positions DevSecOps as a bridge between software assurance and data governance, proposing that automation, Infrastructure-as-Code, and policy enforcement can transform ETL pipelines from operational utilities into verifiable systems of record. The research completed so far defines the regulatory context, design requirements, and evaluation framework that will guide the subsequent construction and assessment of the DevSecOps-enabled pipeline.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

In recent years, the value of data has surpassed that of the software systems built to process it [27,37]. Organisations increasingly treat data as a product, complete with ownership structures, stewardship responsibilities, and lifecycle management frameworks [27]. This shift has coincided with growing regulatory scrutiny across jurisdictions [27]. Frameworks such as APRA's Prudential Standards [5–8], alongside global instruments such as the EU's GDPR and the United States' HIPAA, demand not only the secure handling of information assets but also transparent processes that demonstrate accountability and control [37].

Extract–Transform–Load (ETL) workflows lie at the heart of this transformation, serving as the operational backbone of analytical systems [36, 38]. As Mahmud and Ikbal [22] note, modern ETL pipelines have evolved into socio-technical infrastructures that sustain the scalability, reliability, and legitimacy of data-driven decision-making. Yet despite the rise of DataOps, many pipelines remain opaque, lacking end-to-end lineage, reproducibility, and verifiable controls [37]. In regulated environments, such opacity undermines compliance and erodes institutional trust.

Software engineering has already faced similar challenges. DevSecOps — the integration of development, security, and operations — has matured into a discipline embedding verification, automation, and governance across the software delivery lifecycle [3, 4, 10, 28]. If data is now the product, DevSecOps offers an architectural and cultural foundation for treating data pipelines with the same rigour once reserved for software systems [34]. This research explores that proposition: how DevSecOps principles can be adapted to strengthen auditability and data governance within ETL workflows.

1.2 Problem Statement

While the DataOps movement has improved automation and observability, its primary focus remains operational efficiency rather than regulatory assurance [22,25]. Existing orchestration frameworks, though powerful, rarely produce immutable audit trails, reproducible execution states, or cryptographically verifiable lineage. Auditability thus emerges as an accidental property rather than a designed one [39,40]. As a result, organisations face a persistent gap between the technical execution of pipelines and the evidentiary requirements of regulators [6,8].

This thesis addresses that gap. It investigates how a *DevSecOps-enabled toolchain* can enhance auditability and data governance across the ETL lifecycle. DevSecOps has already institutionalised security and compliance for software artifacts, but an equivalent framework for data artifacts has yet to emerge. This absence exposes regulated organisations to compliance risk, weakens governance, and impairs their ability to demonstrate trustworthy data management.

1.3 Research Objectives

This research adopts a Design Science methodology [17] to design, implement, and evaluate a DevSecOps-based toolchain that enhances auditability and governance in ETL workflows. The study follows an iterative cycle of artifact design, prototype construction, and evaluation against explicit metrics for lineage completeness, reproducibility, and tamper detection.

Primary Research Question

> How can a DevSecOps toolchain be designed to support ETL workflows in regulated industries, enhancing auditability and data governance?

In the process of answering the above question, this paper explores five sub-questions:

Sub-Questions

1. How do ETL workflows manifest in regulated industries, and what auditability and data governance challenges do they face?
2. How do DevSecOps practices enhance auditability and data governance within the software development lifecycle?
3. In what ways do data lifecycle management practices extend or reshape the software development lifecycle when DevSecOps principles are applied to ETL workflows in regulated environments?

4. What categories of tooling exist in DevSecOps and ETL pipelines, and how do their features support auditability and data governance?

5. How can the effectiveness of a DevSecOps-enabled ETL toolchain be evaluated in terms of auditability and data governance?

1.4 Contributions

This thesis makes three primary contributions. First, it proposes an **architectural model** for a DevSecOps-enabled ETL toolchain, aligning the data lifecycle with secure software delivery practices to improve auditability and governance. Second, it presents a **prototype implementation** that integrates infrastructure as code, automated security scanning, and immutable logging into an operational ETL environment built with Airflow, Spark, and AWS components. Third, it defines an **evaluation framework** for characterising auditability and governance through traceability, reproducibility, and tamper-resistance criteria, directly addressing Sub-Question 5. Together, these contributions demonstrate how DevSecOps principles can be embedded in data-centric systems to enhance compliance and trust.

1.5 Project Timeline

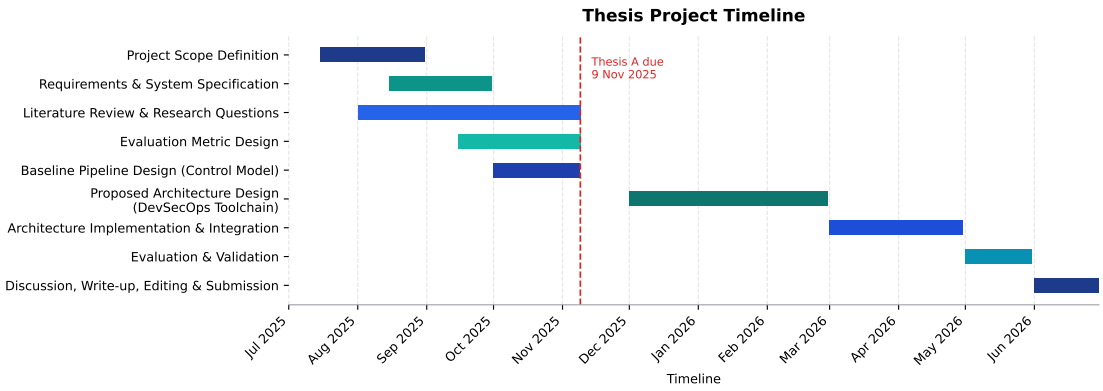

Figure 1.1: Gantt chart of project timeline.

1.6 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2** reviews literature on ETL pipelines, data governance, and DevSecOps practices.

- **Chapter 3** explains the Design Science methodology, design constraints,and evaluation criteria.
- **Chapter 4** describes the baseline ETL pipeline and identifies auditability and governance gaps.
- **Chapter 5** presents the proposed DevSecOps-integrated architecture.
- **Chapter 6** presents the implementation results, including the governed runtime evidence, manifests, versions, and audit reconstruction example.
- **Chapter 7** evaluates the system against the auditability and governance criteria defined in Chapter 3, addressing Sub-Question 5.
- **Chapter 8** discusses implications, evaluation scope, and avenues for future research.
- **Chapter 9** concludes with reflections on how DevSecOps can enable trustworthy data governance in regulated environments.

The results for this thesis are split between the evaluation framework found in Chapter 3, the baseline ETL pipeline described in Chapter 4, and the governed prototype results reported in Chapter 6.

Chapter 2

Literature Review

This chapter reviews the body of knowledge relevant to the design of ETL pipelines in regulated industries and the application of DevSecOps principles. It begins by outlining the context of ETL and governance, defines the key concepts central to this research, and then examines the literature associated with each research question.

2.1 Background

Data pipelines underpin modern analytical systems, forming the connective infrastructure between data sources and analytic platforms [34]. They transform raw data into governed, traceable data products used in applications such as fraud detection, market analysis, compliance reporting, and metadata processing.

Although the extract–transform–load (ETL) process appears simple in theory, it is in practice complex, resource-intensive, and often the most time-consuming element of data warehousing [18]. ETL pipelines act as the "eyes of the system," feeding reliable data downstream; when lineage is lost or visibility fails, the reliability of the entire process is compromised [36]. Vayyala [36] emphasises that maintaining end-to-end lineage across transformations is vital for sustaining trust in analytics-driven decisions.

In regulated industries, compliance depends as much on how data is processed as on the outcomes produced. Frameworks such as GDPR, SOX, and APRA's CPS standards require secure, ethical, and well-documented data handling [5–8, 27]. These frameworks place accountability, ownership, and an end-to-end evidence chain at the centre of compliance culture.

Auditability, in this context, is the ability to demonstrate compliance through reproducibility, lineage, and tamper-evident records [37]. Trustworthy analytics rely not only on integrity of data but also on assurance evidence of correct process execution. Governance provides the organisational scaffolding for this assurance—linking policy to technical enforcement through defined roles, procedures, and controls [27, 37]. Failures in either dimension expose organisations to legal,

financial, and reputational risk [27].

DevOps emerged to accelerate software delivery through automation, continuous integration and deployment (CI/CD), and rapid feedback cycles [28]. DevSecOps extends this model by embedding security and compliance across the software development lifecycle. Practices such as automated testing, compliance-as-code, artifact signing, and tamper-evident logging produce assurance evidence that can be inspected by regulators [4, 10, 28]. This convergence bridges engineering and governance concerns, establishing control assurance as a built-in property of the delivery process.

As data itself becomes a primary strategic asset, surpassing the value of the systems that process it [27, 37], the mechanisms of DevSecOps must be extended to the data pipelines that sustain organisational decision-making.

2.2 Defining Key Concepts

The following subsections define the key concepts underpinning this research: DevSecOps, ETL, data governance, and auditability. Each definition draws from existing literature and sets the theoretical foundation for later analysis.

DevSecOps

DevSecOps is the proactive and continuous enforcement of security and compliance requirements throughout the software development lifecycle (SDLC) [4]. Unlike traditional models that treat security as an external stage or afterthought, DevSecOps embeds it within every phase of the lifecycle [3]. Compliance becomes the expected outcome by default, and deviations from defined standards must be explicitly justified [10].

DevSecOps is notoriously difficult to define succinctly. It is often described as the sum of its implementation patterns—CI/CD, infrastructure-as-code (IaC), policy-as-code (PaC), software scanning, automated tests, and GitOps. A more practical lens is to examine the gap created when DevSecOps is removed: the ungoverned space between code that compiles and a production service. In that gap, identity and approval are ad hoc, policy enforcement is manual, change control is episodic, evidence is incomplete, and configuration drifts without reconciliation. DevSecOps closes this gap by making control assurance continuous, placing verification at the point of change, and generating the lineage and evidence metadata needed for independent assessment [4, 33].

A defining characteristic of DevSecOps is its reliance on automation to enforce non-functional requirements such as control assurance and governance. Through automated testing, provisioning, monitoring, and policy enforcement, systems produce assurance evidence—signed artifacts, tamper-evident records, and control indicators—that regulators and auditors can independently assess [4, 44]. This

automated assurance narrows the gap between delivery teams and governance stakeholders [3].

By combining cultural expectations of shared responsibility with technical mechanisms for verification, DevSecOps establishes a delivery model in which security and compliance are continuous, measurable, and intrinsic—rather than externally imposed.

ETL

Extract–Transform–Load (ETL) is a structured process that ingests, transforms, and persists data into target repositories such as data warehouses, lakes, or file stores [18]. It traditionally comprises three stages: extraction, where data is sourced from heterogeneous systems; transformation, where it is cleansed, standardised, integrated, and reshaped into coherent semantic structures; and loading, where transformed data is stored for downstream analysis [18]. Kimball and Caserta describe ETL as both the most resource-intensive and most critical link between operational data and analytical use.

Modern practice extends beyond this classical batch model. Approaches such as extract–load–transform (ELT) and hybrid streaming–batch architectures leverage distributed and cloud-native infrastructure [22]. ETL now encompasses not only data integration and quality but also governance and compliance. In regulated industries, pipelines must guarantee lineage, reproducibility, and control assurance in addition to performance and scalability [22].

ETL is thus best understood as a flexible socio-technical process. It concludes when data is made persistent and reusable but intersects with broader lifecycle concerns such as orchestration, monitoring, and policy enforcement. In this sense, ETL pipelines underpin both analytical scalability and institutional trust [22].

Data Governance

Data governance provides the strategic framework that defines how data is collected, managed, and protected across an organisation. It establishes the policies, roles, standards, and processes that ensure information assets remain accurate, consistent, secure, and compliant with internal and external regulations [27, 37]. Nai, Rifai, and Sadiq [27] describe governance as a structured framework that ensures the availability, integrity, usability, and security of data through stewardship and accountability roles.

Vayyala [37] expands this understanding by positioning governance as the foundation of reliable data ecosystems—linking policy to technical enforcement through automation, lineage tracking, and tamper-evident records. In practice, governance institutionalises accountability so data products can withstand regulatory scrutiny.

Auditability

Auditability is the designed capability of a system to allow its processes, controls, and outcomes to be observed and verified by stakeholders. Weigand and Elsas [40] define auditability through five interdependent requirements:

- **Normativity**: a clear regulatory or contractual framework that governs operations.
- **Accountability**: verifiable statements linking actions to responsible agents and governing norms.
- **Referentiality Infrastructure**: records that connect operations to their original points of recording.
- **Reliability**: faithful and tamper-resistant representation of events.
- **Verifiability**: the ability of external parties to independently test or validate claims.

Although these principles originate in organisational and financial control, they can be transferred to information systems. Extending this theoretical foundation, Wang et al. [39] demonstrate how auditability can be operationalised through third-party verification, cryptographic proofs of integrity, and support for dynamic data operations. Their work illustrates that auditability is not merely an organisational goal but a property that can be engineered into data management infrastructures.

Applied to ETL pipelines and DevSecOps practices, these five dimensions map directly to regulated-industry concerns:

- **Normativity**: embedding compliance and data-protection policies within workflow execution.
- **Accountability**: linking each transformation or deployment to authenticated agents or automated processes.
- **Referentiality Infrastructure**: ensuring full lineage tracking across extraction, transformation, and loading.
- **Reliability**: maintaining tamper-evident records, signed artifacts, and cryptographic hashes.
- **Verifiability**: enabling reproducibility tests and independent control testing via CI/CD-integrated checks.

Together, these adapted requirements frame auditability as a designed and engineered property of ETL systems. Every data operation and system change must attach to an end-to-end evidence chain that is reproducible and independently verifiable against regulatory expectations.

2.3 RQ1: ETL Pipelines in Regulated Industries

ETL pipelines in regulated industries operate within a constant tension between agility and accountability. They are expected to deliver analytical value with speed and scale, yet must simultaneously produce verifiable evidence that every data movement complies with institutional and regulatory constraints. The modern pipeline has therefore evolved into a socio-technical infrastructure—one that mediates trust between engineers, auditors, and regulators [22,36]. Its success cannot be judged solely by throughput or latency; rather, by its ability to maintain lineage, transparency, and compliance under continuous change.

Financial, healthcare, and critical-infrastructure systems exemplify this dual demand. Frameworks such as APRA's CPS 230, CPS 234, and CPS 510 [6–8] institutionalise separation of duties and controlled promotion of change, translating ethical responsibility into technical procedure. These standards do not exist apart from engineering practice—they shape it—embedding legal accountability into operational design. Governance scholarship reaches similar conclusions, emphasising that tamper-evident logging, role-based accountability, and retention aligned with compliance lifecycles form the practical machinery of institutional trust [27, 37, 38]. Cloud-native orchestrators such as Airflow, AWS Glue, and Azure Data Factory provide elasticity and automation, but they operate under the shadow of these mandates. Their convenience is constrained by the need to preserve traceability and access control consistent with CPG 235 [5]. In this context, orchestration itself becomes a form of governance: a choreography of tasks and dependencies that leaves a verifiable trail of execution [31].

Over time, this operational discipline has turned the pipeline from a means of data transport into a living archive of control assurance. Continuous validation, versioning, and policy enforcement integrate assurance logic directly into the runtime environment [37]. Metadata management forms the connective tissue between policy and process, ensuring that lineage and evidence metadata, schema evolution, and transformation logic can be reconstructed and defended [38]. Yet even within mature systems, full control assurance remains elusive. Toolchain heterogeneity fragments lineage across layers of orchestration, computation, and storage. Business logic embedded in SQL or user-defined functions escapes formal tracing, while schema drift and inconsistent metadata propagation corrode reliability [22]. Few frameworks create tamper-evident, cryptographically verifiable artifacts by default [38,40]. What is missing is not information, but integrity—the ability to prove that recorded operations are both complete and untampered. The fragility of this end-to-end evidence chain provides the practical motivation for integrating DevSecOps assurance into ETL environments, turning validation from an external audit to an intrinsic system property.

2.4 RQ2: DevSecOps and Governance in the SDLC

The software development lifecycle (SDLC) treats change as a controlled and observable process. Standards such as ISO/IEC/IEEE 12207 and 15288 define the artifacts, procedures, and documentation that make each phase accountable [1,16]. Governance in this model is not an external overlay—it is embedded in the way systems are planned, built, and released.

DevSecOps strengthens this principle by embedding assurance, security, and compliance across every phase of delivery [4, 33]. The delivery pipeline becomes a generator of assurance evidence: each build, test, and deployment produces signed artifacts, tamper-evident records, and compliance manifests that bind design intent to execution [3,4,11,40]. These artifacts collectively form a continuous end-to-end evidence chain that makes system state traceable and verifiable at any point in time.

Automation enables this continuity. Infrastructure-as-code (IaC), continuous-integration/continuous-delivery (CI/CD), and policy-as-code (PaC) apply verification directly at the point of change. Static and dynamic testing, dependency scanning, and approval gates transform compliance into an embedded control validation signal rather than a periodic audit [3, 16, 21]. The NIST Secure Software Development Framework codifies this expectation: traceable metadata, lineage and evidence metadata, and policy enforcement are standard lifecycle outputs [33]. Mature build systems act as lineage and evidence-metadata engines, linking requirements, commits, and runtime behaviour within the same evidentiary record.

As software supply chains expand, continuous verification of component origin and integrity becomes essential [21, 33]. Toolchains respond through automated scanning, provenance analysis, and AI-assisted policy enforcement [2, 23]. Requirement traceability then links regulatory expectations directly to runtime artifacts [1, 11], ensuring that what is built, deployed, and reviewed aligns to the same authoritative record [28].

Several implementation patterns operationalise DevSecOps principles of automation and assurance. Infrastructure-as-Code (IaC) enforces reproducible, governed environments; Policy-as-Code (PaC) embeds compliance validation within delivery pipelines; and GitOps extends these mechanisms into runtime operations by reconciling deployed state with version-controlled configuration. Together, these patterns close the loop between design-time verification and operational control, forming the practical foundation of continuous control monitoring within the SDLC.

2.5 RQ3: Comparing the Data Lifecycle and the SDLC

The data lifecycle (DLC) aims for control and reliability but lacks the SDLC's formal standardisation. Wing describes the DLC as a dynamic, context-dependent model rather than a fixed sequence of phases [41]. Most frameworks share the same goal: to keep data reliable, accessible, and verifiable throughout its lifespan [9,20]. The SDLC governs change and transformation; the DLC governs persistence and stewardship. One manages evolution, the other preserves continuity.

Despite their structural differences, the two lifecycles can be aligned. The Data Management Body of Knowledge (DAMA-DMBOK) provides one of the most widely adopted codifications of the DLC, framing governance through roles, metadata catalogues, lineage records, and quality assessments that together establish an end-to-end evidence chain [13]. These governance artifacts closely mirror the information items defined in ISO/IEC/IEEE 15289 [1], demonstrating that the logic of software assurance maps naturally onto data management practice. DevSecOps supplies the mechanisms for this connection: continuous validation and automated security controls apply equally to code and data [4,33]. Lineage manifests, schema definitions, and transformation scripts can therefore be versioned and audited alongside software components.

Encryption, secrets management, and role-based access add cryptographic assurance to this framework [29,34]. Agile governance pushes data validation earlier in the lifecycle [19, 36, 38], mirroring the shift in DevSecOps toward early verification. These expectations reflect APRA's prudential standards [5–8], where assurance is evidenced through the operational trace rather than through static reports.

When the SDLC and DLC intersect, they create a shared evidentiary framework. ISO/IEC/IEEE 15289's artifacts—plans, specifications, procedures, and reports—extend to include lineage documentation, schema evolution, and validation outputs. IaC and policy-as-code make these records executable [3, 26, 28]. Reproducibility, tamper detection, and governance then operate through the same automated pipelines that deliver software.

In regulated ETL workflows, control assurance becomes a first-class design property. Each transformation of code or data leaves a signed, tamper-evident record that shows what occurred, who performed it, and why it was authorised.

2.6 RQ4: Tooling for Control Assurance and Governance

Assurance depends on implementation. The ability to trace, verify, and reproduce each operational event relies on tools that enforce control, capture lineage, and preserve evidence of change. In both DevSecOps and ETL environments, tooling

translates regulatory requirements such as those in CPG 235 and CPS 234/510 into machine-verifiable artefacts [5–7].

This section examines the tool categories that make such evidence possible. In DevSecOps, frameworks for infrastructure-as-code, policy validation, and continuous integration generate signed artefacts and lineage and evidence metadata that prove system integrity. In ETL pipelines, ingestion and orchestration components, processing engines, metadata catalogues, validation frameworks, and monitoring systems create traceable datasets and transformation histories that support data governance. Each tool category contributes to the same end-to-end evidence chain, linking software and data processes into a single, verifiable system of record.

DevSecOps tooling: from code to operations

Infrastructure- and Policy-as-Code

IaC and PaC convert infrastructure and policy into versioned artefacts so that compliance checks run before change. Terraform, CloudFormation, OPA, Checkov, and Conftest generate explicit evidence of desired state and policy evaluation [4, 29]. Key contrasts appear in Table 2.1.

Table 2.1: IaC/PaC tools and the evidence they produce

Tool	Strengths	Limitations
Terraform	Versioned infra; plan/apply logs; OPA integration	Limited audit trail
CloudFormation	CloudTrail linkage; tight IAM controls	Vendor lock-in
OPA / Conftest	Declarative policy; decision logs as evidence	Rule complexity

CI/CD orchestration

GitHub Actions, GitLab CI, Jenkins, and Azure Pipelines attach identity, review, and signatures to every run; pipeline logs form part of the evidentiary chain [11, 33]. Table 2.2 summarises key trade-offs.

Table 2.2: CI/CD systems as provenance engines

Tool	Strengths	Limitations
GitHub Actions	Signed runs and artefacts; Git provenance	Coarse access audit
GitLab CI	Built-in approvals; compliance reports	Higher operational cost
Jenkins	Extensible audit plugins	Plugin sprawl
Azure Pipelines	RBAC tie-in; policy checks	Cloud-specific integrations

SAST, DAST, and SCA

Security scanners create structured evidence of vulnerabilities and dependency lineage. They integrate with CI/CD pipelines and feed policy gates [12, 29, 32]. Table 2.3 summarises trade-offs.

Table 2.3: Security scanning tools and audit artefacts

Tool	Strengths	Limitations
SonarQube	Tracks code quality and issues	Limited container context
OWASP ZAP	Reproducible web tests; open output	Slow at scale
Snyk	Dependency risk scoring; SBOMs	Proprietary backend
Trivy	Combined SCA/SAST; SBOMs	Narrow policy scope

Testing and validation

Test frameworks provide repeatable proof of expected behaviour; their outputs become verifiable artefacts that link requirements to executions [32,33]. Table 2.4 outlines key examples.

Table 2.4: Testing frameworks as reproducible evidence

Framework	Strengths	Limitations
Postman / Newman	Versioned API suites; runnable logs	Manual upkeep of collections
PyTest	Parametrised fixtures; stable outputs	Python-only scope
JUnit	Mature CI integration; clear reports	No native audit metadata

Containers and runtime

Images, admission controls, and runtime events contribute directly to system integrity evidence [4, 28]. Table 2.5 lists key tools and their audit characteristics.

Table 2.5: Container and runtime controls as integrity evidence

Tool	Strengths	Limitations
Docker	Image signatures; digest verification	Limited runtime policy control
Kubernetes	Declarative policy; drift detection	Complex multi-cluster audits
Falco	Real-time event monitoring	High rule upkeep
Prisma Cloud / Sysdig	Centralised compliance view	Proprietary; costly

Monitoring and observability

Control indicators and logs form the final evidence layer, supporting incident reconstruction and trend analysis [4, 29]. Table 2.6 highlights typical stacks.

Table 2.6: Observability stacks and audit support

Tool	Strengths	Limitations
Prometheus	Transparent, timestamped metrics	High cost for long retention
Grafana	Clear dashboards; evidence views	Dependent on source data quality
ELK / Splunk	Searchable forensic logs	Operational cost; index complexity

ETL tooling: from ingestion to lineage

ETL ecosystems expose their own audit surface. Ingestion tools capture how data enters governed infrastructure; orchestrators record how it moves; processing engines determine how it changes; and metadata, validation, privacy, and monitoring systems preserve the story of what happened and when. Together, these tools produce the evidence needed to link data handling to governance expectations in regulated environments [22, 35, 38, 42].

Orchestration

Orchestrators manage dependencies, retries, and execution order. Their run histories and task states give ETL workflows a temporal structure that can be replayed and inspected [22, 35]. They connect ingestion with transformation and expose where control decisions were applied. Table 2.7 compares common orchestration systems and the forms of execution evidence they provide.

Table 2.7: Orchestration systems and their execution evidence

Tool	Strengths	Limitations
Apache Airflow	Clear DAGs; task history	Manual scaling; weak lineage
Azure Data Factory	Built-in control; Purview link	Limited cross-cloud use
AWS Step Functions	Visual flows; CloudWatch logs	Coarse logs; AWS-only
Dagster	Data-aware; built-in lineage	Small community; steep setup

Transformation engines

Processing engines carry out the transformations that regulators most care about; they decide how raw records become derived data products. Engines such as Spark, EMR, Dataflow, and Delta Lake can provide deterministic checkpoints

and time-travel capabilities that support reproducible proofs of change [22, 24]. Table 2.8 outlines their main assurance properties.

Table 2.8: Processing engines and reproducibility guarantees

Engine	Strengths	Limitations
Apache Spark	Distributed compute; checkpoint versioning	Complex lineage tracing
Google Dataflow	Managed scaling; audit logging	Cloud lock-in
Delta Lake	ACID tables; rollback support	Storage overhead

Metadata and lineage

Catalogues and lineage systems form the spine of auditability: they connect datasets, schemas, and processes into a coherent graph that can be queried and tested [38, 42]. These tools turn implicit relationships into explicit evidence, supporting both governance and impact analysis. Table 2.9 summarises key options.

Table 2.9: Metadata and lineage systems as audit structure

Tool	Strengths	Limitations
Apache Atlas	Open lineage; policy integration	Latency at scale
Microsoft Purview	Rich lineage views; Azure native	Proprietary scope
Amundsen	Lightweight metadata discovery	Limited enforcement links

Operational monitoring

Monitoring and observability tools turn runtime behaviour into an audit surface. Metrics, traces, and logs enable lineage verification, incident reconstruction, and control testing over time [35]. In ETL contexts, these systems complete the evidence chain by linking pipeline events to infrastructure and application signals. Table 2.10 summarises common options.

Table 2.10: ETL observability as an audit surface

Tool	Strengths	Limitations
Azure Monitor	Unified infra and data view	Short retention limits
AWS CloudWatch	Deep AWS integration; trace logging	Complex audit queries
Prometheus	Open telemetry; clear metrics	High data volume

Convergence

GitOps as an Operational Assurance Pattern

GitOps is a practical implementation of DevSecOps principles that operationalises configuration and change assurance. By declaring desired system state in version control and automating reconciliation, GitOps enforces configuration provenance, segregation of duties, and continuous control validation across environments. In this unified model, the same controls that attest software integrity also attest data integrity. Approval gates in CI/CD govern the promotion of ETL workflows; policy checks constrain data transfers; and container attestations record the runtime context of both applications and transformations. Observability systems link these artifacts into a continuous audit timeline spanning code, infrastructure, and data [4,15,33]. GitOps enforces configuration provenance; complementary lineage controls extend this assurance to data artifacts and transformations.

These implementation patterns—particularly GitOps—form the operational basis of the proposed architecture in Chapter 5, where declarative configuration and automated reconciliation underpin continuous assurance across both software and data lifecycles. Regulatory expectations under CPG 235 and CPS 234/510 [6, 7] are satisfied not through separate audits but through a single, declarative lifecycle.

Three assurance patterns make this convergence practical. *Single-path promotion* ensures that every artifact—application, configuration, or dataset—follows the same reviewed path to production. *Policy-guarded reconciliation* binds control objectives to each applied change. *Attested runtime* couples container and orchestration evidence to specific commits and policies, closing the loop between declared intent and observed outcome [4, 33].

2.7 RQ5: Evaluating Auditability and Governance

Evaluation forms a core component of Design Science Research, providing the means to demonstrate an artifact's utility, quality, and validity in context. Hevner [17] and Peffers [30] describe evaluation as integral to the build–evaluate cycle, while Gregor and Hevner [14] distinguish between *ex-ante* evaluation—assessing design logic before implementation—and *ex-post* evaluation, which measures performance in operation. Together, these works establish evaluation as both a validation activity and a source of knowledge in its own right.

Within DevSecOps literature, evaluation typically focuses on security or compliance performance rather than evidentiary assurance. Souppaya and Scarfone's NIST SSDF [33] and Anisetti et al. [4] present process metrics for secure software delivery, while Weigand and Elsas [40] frame auditability as a design property requiring verifiable, tamper-evident records. These studies demonstrate that control assurance can be measured, but they stop short of applying such evaluation to

data-intensive workflows that must also satisfy regulatory expectations for lineage, reproducibility, and provenance.

From a data-engineering perspective, evaluation of auditability is most often addressed indirectly through lineage reconstruction, data quality assessment, and provenance capture [22, 36, 38]. ETL research typically measures performance in terms of latency, scalability, or transformation accuracy, while evidence of compliance and reproducibility is treated as secondary. Existing frameworks for data lineage and metadata management establish mechanisms for tracing data flow but rarely define how the completeness or trustworthiness of that lineage should be evaluated [39]. As a result, few approaches combine the quantitative rigor of DevSecOps metrics with the contextual assurance required in regulated data environments.

The literature therefore points to a gap between existing DevSecOps evaluation frameworks—focused on software artifacts—and the needs of data-governance environments, where evidentiary completeness and verifiable lineage are central. Addressing this gap requires extending evaluation beyond security metrics to encompass measurable dimensions of auditability and governance in data-centric systems.

2.8 Synthesis and Related Works

Research at the intersection of DevSecOps and data governance converges on the same foundation—automation, traceability, and verifiable control—but few works treat these as components of a unified evidence system spanning the full ETL lifecycle. Existing frameworks address individual facets of the assurance problem—data quality, lineage, or security—but seldom integrate them under a common governance architecture capable of satisfying regulatory expectations for auditability.

The IGI Global collection *Data Governance, DevSecOps, and Advancements in Modern Software* [27] provides the strongest conceptual foundation. Its chapters illustrate how metadata management, lineage, and policy-as-code can bridge security automation and governance accountability. Vayyala [38] presents metadata as a governance mechanism; Kose [19] links agile compliance to DevSecOps; and Soussane [34] applies these principles to real-time analytics. Collectively, they demonstrate how DevSecOps can embed governance within delivery pipelines, though they stop short of implementing tamper-evident and reproducible evidence across the data lifecycle.

Subsequent studies in software assurance and lifecycle management reinforce these ideas but remain domain-bound—software or data, not both. Anisetti et al. [4] and Souppaya and Scarfone [33] formalise metrics for secure software delivery, while Wing [41] and Zhang et al. [43] explore stewardship and reliability within data lifecycles. Yet no current model provides a framework that spans both domains to create a single, tamper-evident audit trail linking code, configuration,

and data artifacts.

The research gap is therefore twofold. First, auditability remains treated as an emergent property rather than a designed capability of ETL systems. Second, evaluation methods in both DevSecOps and data governance literature rarely consider evidentiary completeness or lineage trustworthiness as measurable outcomes. This thesis extends the existing body of work by proposing a unified DevSecOps-enabled ETL toolchain that operationalises continuous assurance through verifiable lineage, reproducibility, and governance evaluation.

Chapter 3

Methodology

This chapter outlines the methodology used to design, construct, and evaluate a DevSecOps toolchain that governs ETL workflows in regulated environments, including the framework for evaluating auditability and governance defined in Sub-Question 5.

The research applies a Design Science Research (DSR) approach, chosen for its focus on building artifacts that address real-world problems while generating transferable knowledge [17]. Rather than developing a new ETL pipeline, this study designs the system that governs such pipelines—the architecture, control automation, and controls that embed assurance and compliance into operational data workflows.

3.1 Design Science Context

Design Science generates knowledge through the purposeful creation and evaluation of artifacts [30]. It follows an iterative build–evaluate process in which understanding arises from constructing, implementing, and refining solutions. In this study, the artifact is a *DevSecOps governance and assurance framework*—a set of integrated components that enforce policy, record lineage and evidence metadata, and assure integrity across an ETL pipeline. The pipeline itself functions as a *reference pipeline* that represents a typical control-baseline environment within a regulated domain.

This methodological choice is appropriate because Design Science balances *rigour*—alignment with regulatory and theoretical principles—with *relevance*—demonstrated utility in practice. Each design iteration: problem identification, artifact construction, demonstration, and evaluation, corresponds to a development stage where the framework's assurance features are incrementally refined and tested. This cyclical process mirrors the continuous control monitoring model advocated in DevSecOps, where systems evolve through assurance evidence and feedback.

3.2 Regulatory Context and Requirement Derivation

The toolchain design is guided by the Australian Prudential Regulation Authority's (APRA) Prudential Standards: CPS 230 (*Operational Risk Management*), CPS 234 (*Information Security*), and CPS 510 (*Governance*) [6–8]. These standards define what regulated entities must achieve in governance, security, and resilience, but they do not prescribe how those objectives should be implemented technically. Their descriptive, outcomes-based language demands interpretation into enforceable engineering controls.

Although the methodology is domain-agnostic, this research focuses on the *financial sector*. Finance offers a mature and well-documented regulatory landscape that provides both conceptual depth and practical relevance for governance evaluation. APRA's prudential standards form a coherent, enforceable set of requirements that align with the objectives of assurance, lineage, and accountability identified in the literature. This focus reflects the local regulatory environment and the availability of structured compliance expectations that make the sector suitable for Design Science evaluation.

Accordingly, the scope is narrowed to a practical subset of CPS requirements most relevant to data-pipeline governance. The selected principles—continuity, accountability, and control—are translated into implementable requirements for control automation and evidence generation. This refinement is informed by the literature review in Chapter 2, which frames auditability (as defined there) in terms of lineage, evidence, and verification across socio-technical systems.

3.3 Requirements

The following requirements define what the DevSecOps framework must achieve to operationalise prudential expectations. They establish the conditions under which the framework can govern a pipeline by embedding control, lineage, and continuous assurance.

1. **R1: Governance Accountability (CPS 510).** The framework must preserve authorship and approval lineage for all operational and configuration changes, ensuring full traceability of decision authority and execution.

2. **R2: Information Security Assurance (CPS 234).** The framework must maintain artifact integrity and confidentiality through tamper-evident storage, cryptographic verification, and continuous security scanning (SCA, SAST, and DAST).

3. **R3: Operational Risk Control (CPS 230).** The framework must detect, record, and remediate process or configuration failures, maintaining reproducibility and recovery within defined assurance thresholds.

4. **R4: Continuous Control Monitoring (CPS 234, 510).** The framework must automate policy validation and control testing in the CI/CD process, producing machine-verifiable assurance evidence through policy-as-code enforcement.

5. **R5: End-to-End Evidence Chain (CPS 230).** The framework must generate and retain a tamper-evident, queryable record that captures the lineage of code, configuration, and data artifacts throughout the system lifecycle.

These requirements shift focus from the business logic of ETL processes to the *meta-layer* of assurance—how evidence is produced, tested, and preserved. They align with the five auditability dimensions defined in Section 2.2: normativity, accountability, referentiality, reliability, and verifiability.

Table 3.1: Mapping between toolchain requirements and auditability dimensions.

Requirement	Normativity	Accountability	Referentiality	Reliability	Verifiability
R1 Governance Accountability	●	●			
R2 Information Security Assurance				●	●
R3 Operational Risk Control			●	●	
R4 Continuous Control Monitoring	●	●			
R5 End-to-End Evidence Chain			●		●

Governance and control monitoring (R1, R4) express the normative and accountable aspects of control; security and risk management (R2, R3) reinforce reliability and traceability; and the end-to-end evidence chain (R5) provides verifiable, referential links across artifacts and executions. Together, these create a governance and assurance framework where assurance is continuous and evidence is intrinsic.

3.4 Evaluation Framework

Evaluation in Design Science verifies that an artifact fulfils its intended purpose. In this study, evaluation operationalises Sub-Question 5 by defining how the DevSecOps framework will be assessed against the requirements R1–R5. For this study, evaluation criteria are expressed qualitatively rather than as fixed numeric thresholds. Each requirement becomes an evaluation unit with observable control indicators, data sources, and assurance conditions, while quantitative benchmarks are deferred to Thesis B once empirical measurements are available.

R1 Governance Accountability

Objective: Confirm that all changes are traceable to authenticated identities and documented approvals.

Method: Examine change events, approval records, and version metadata for completeness and signature validity.

Control Indicators:

- Configuration and code changes are traceable to verified identities.
- Approval events are associated with tamper-evident artifacts.
- Lineage between versions remains continuous and reconstructable.

Auditability Dimensions: Normativity, Accountability.

R2 Information Security Assurance

Objective: Demonstrate that artifacts and configurations are verified for integrity and security.

Method: Conduct automated and manual reviews confirming that artifacts are signed, validated, and free from known vulnerabilities.

Control Indicators:

- Artifacts are verified for integrity through signatures or checksums before use.
- No integrity violations remain undetected or unremediated during testing.
- Verification records are retained and accessible for independent review.

Auditability Dimensions: Reliability, Verifiability.

R3 Operational Risk Control

Objective: Validate reproducibility and recovery from prior workflow states.

Method: Execute controlled replays of historical states and compare outputs to reference baselines.

Control Indicators:

- Lineage information is sufficiently complete to reconstruct mappings between inputs, processing steps, and outputs.
- Historical workflow states can be replayed and produce consistent, explainable results.
- Recovery procedures restore prior states within limits that are operationally acceptable for the reference context.

Auditability Dimensions: Referentiality, Reliability.

R4 Continuous Control Monitoring

Objective: Verify that compliance enforcement and validation operate automatically for every change.

Method: Observe multiple workflow executions to confirm policy checks and validations occur consistently.

Control Indicators:

- Policy checks and control validations run consistently for each change and execution path.
- Unverified or unapproved changes are prevented from reaching production environments.
- Each control evaluation produces verifiable evidence of the checks performed and their outcomes.

Auditability Dimensions: Normativity, Accountability.

R5 End-to-End Evidence Chain

Objective: Confirm that the framework maintains a tamper-evident, queryable record of artifacts and lineage.

Method: Perform end-to-end tracing of artifacts across executions to ensure each output can be reconstructed from recorded evidence.

Control Indicators:

- Evidence and lineage records validate successfully across representative workflow executions.
- Each transformation is traceable to its original inputs and associated configurations.
- No unexplained inconsistencies appear in signature or hash verification during evaluation.

Auditability Dimensions: Referentiality, Verifiability.

Evaluation Synthesis

Each requirement is tested through direct observation and analysis, with results mapped back to the auditability dimensions in Table 3.1. Collectively, these evaluations determine whether the governance and assurance framework establishes a verifiable end-to-end evidence chain across its lifecycle, thereby addressing Sub-Question 5 in the context of the reference pipeline.

Chapter 4

Baseline Pipeline

This chapter defines the baseline pipeline that serves as the control model for later evaluation. It represents a minimal but coherent ETL workflow constructed from standard open-source and cloud-native components, configured without embedded governance or assurance controls. The baseline shows how data moves, where control resides, and what traces of evidence are naturally produced. It establishes the reference point against which the DevSecOps architecture in Chapter 5 will be measured.

4.1 Overview

The baseline pipeline ingests public financial filings from the SEC EDGAR API, stages them as raw CSV objects in Amazon S3, transforms them with PySpark on EMR, and outputs curated Parquet datasets back to S3. Airflow, running on a Dockerised EC2 instance, orchestrates the workflow. The infrastructure is declared in Terraform and deployed through GitHub Actions, which performs linting, builds the Airflow image, provisions resources, and redeploys on each `push` to `main`.

Conceptual scope. The configuration models a typical cloud-native pipeline stripped of security and compliance instrumentation. Defaults are accepted to keep the environment simple and reproducible; the goal is not performance optimisation but clarity. This baseline represents the state of practice before DevSecOps principles are introduced to turn operational consistency into demonstrable assurance.

4.2 Architecture

The system operates across three planes—*orchestration*, *compute*, and *storage*—with automation and infrastructure layers supporting them. Figure 4.1 shows the overall arrangement within the AWS environment. Each component reflects a tooling

category defined in Chapter 2 and illustrates both its operational value and its assurance gap.

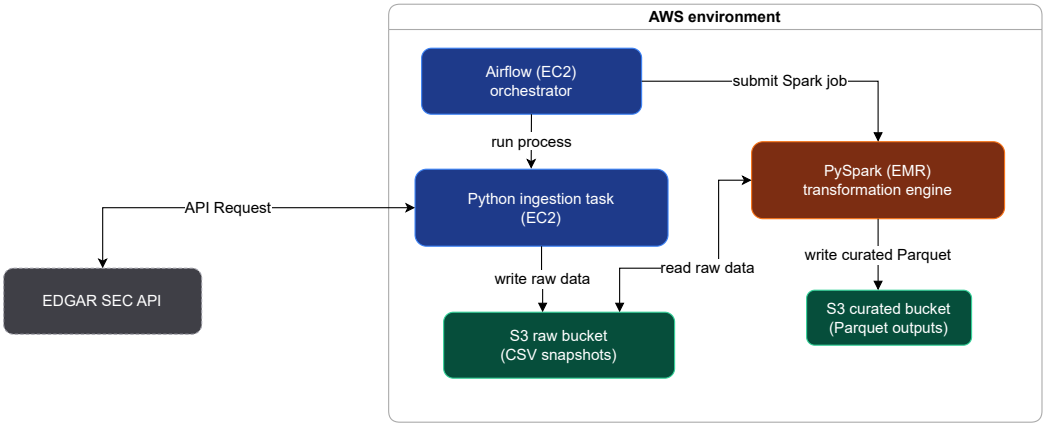

Figure 4.1: Baseline runtime architecture showing orchestration, compute, and storage components.

Orchestration: Airflow. Airflow defines task dependencies and execution order across the ETL process. It embodies the orchestration tools discussed in Section 2.6, functioning as a runtime scheduler and provenance log. In the baseline, these logs are mutable text records with no signatures or policy linkage; they record execution but not authority. The system therefore guarantees temporal order but not accountable control.

Compute: PySpark on EMR. PySpark provides distributed transformation aligned with the compute and runtime control category from Section 2.6. It enforces schema validation and predictable transformations across nodes. Yet job scripts, dependencies, and configurations remain mutable; no digital signatures or versioned attestations confirm authorship. The compute layer executes correctly but offers no cryptographic proof of integrity.

Storage: Amazon S3. S3 acts as both staging and curated store. It ensures durability and timestamped versioning but offers no immutability or content signing. Objects can be modified or deleted without trace, forcing lineage reconstruction from logs rather than metadata. The storage layer is reliable, yet unverifiable.

Infrastructure: Terraform. Terraform declares the infrastructure state and belongs to the Infrastructure-as-Code and Policy-as-Code tooling class. It provides repeatability through declarative configuration but lacks persistence in its execution records; logs are transient and unsigned. The result is infrastructure that can be reproduced, but not proven to match the reviewed plan.

Automation: GitHub Actions. Continuous integration and delivery are handled through GitHub Actions, shown in Figure 4.2. This pipeline unifies code, configuration, and deployment within a single repository; it represents the first step toward a GitOps model. However, it omits policy validation, multi-party approval, and artifact signing. Each push triggers infrastructure updates automatically, without checks that would confirm authenticity or intent.

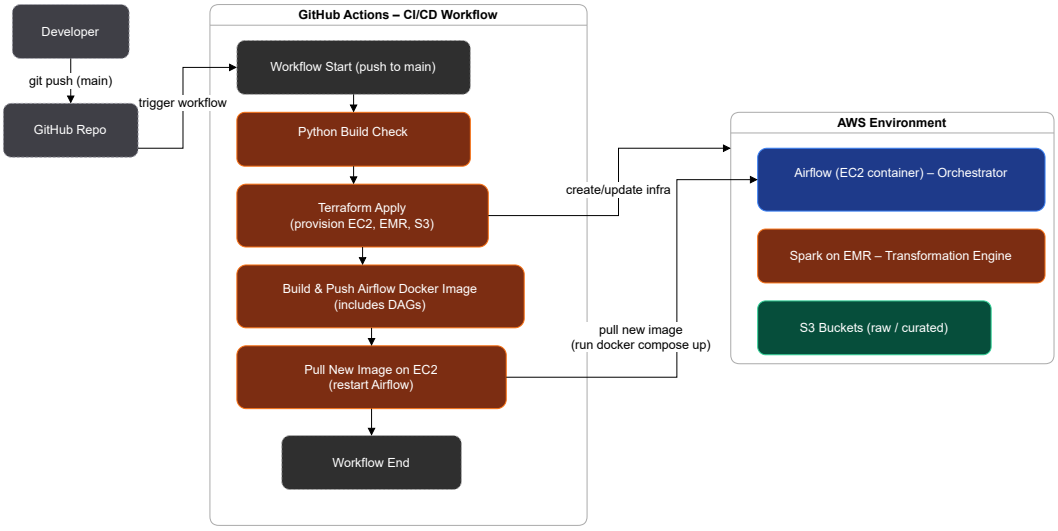

Figure 4.2: CI/CD pipeline implemented in GitHub Actions.

4.3 Functionality

Figure 4.3 shows the Airflow DAG that governs workflow execution. Tasks proceed linearly—ingestion, transformation, and manifest recording. The process runs reliably and can be repeated, but it cannot be independently verified.

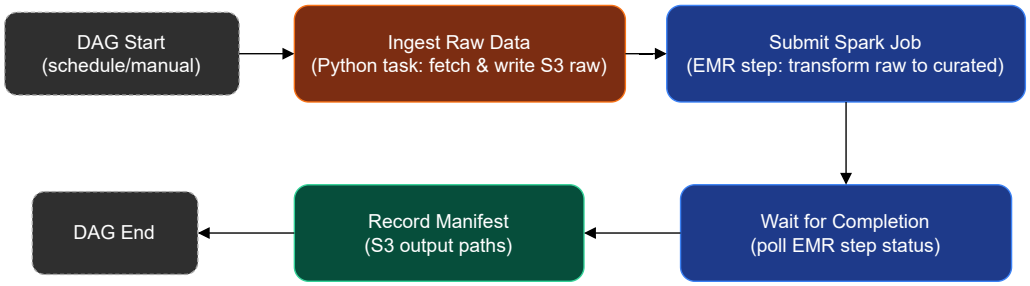

Figure 4.3: Baseline Airflow DAG showing orchestration sequence.

Ingestion. A Python task retrieves structured JSON filings from the EDGAR API, normalises them, and writes timestamped CSV files to a raw S3 bucket. Each run

creates a new partition, forming the input dataset for downstream processing. The representative schema is shown in Table 4.1.

Field	Type	Description
cik	String	Company identifier assigned by the SEC.
form_type	String	Filing category (e.g., 10-K, 8-K).
filing_date	Date	Official submission date.
accession_no	String	Unique accession number for each filing.
file_url	String	Direct URL to the source document.
company_name	String	Registered name of the filer.

Table 4.1: Representative schema of ingested EDGAR filing metadata.

Transformation. The PySpark job reads the raw objects, validates schema consistency, deduplicates, and writes partitioned Parquet datasets to the curated bucket. Logs capture execution time and row counts but omit lineage metadata linking inputs, scripts, and outputs. The pipeline delivers correct results without proving how they were produced.

Output and manifest. A final task generates a `_SUCCESS` manifest in S3, listing output paths for verification. The manifest records completion but not content integrity; no hashes or signatures are included. Figure 4.4 illustrates this flow.

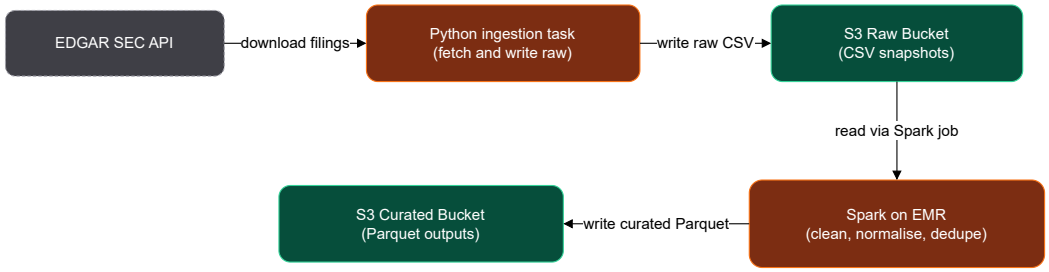

Figure 4.4: Data path from ingestion to curated output in the baseline pipeline.

4.4 Observed Gaps

The pipeline behaves correctly but lacks the mechanisms required to produce credible audit evidence. When compared against requirements R1–R5 from Chapter 3, several weaknesses become clear.

R1: Governance Accountability. Authorship and approval lineage are implicit. Commits identify contributors but are unsigned; Airflow and Terraform

run under shared service identities, leaving no trace between authorised decisions and executed actions. Accountability is assumed rather than enforced.

R2: Information Security Assurance. Infrastructure and data outputs are not verified for integrity. Containers and S3 objects are mutable; dependency scanning is absent. Runtime isolation exists, but cryptographic assurance does not. Evidence of reliability depends on logs that can be altered or lost.

R3: Operational Risk Control. Reproducibility relies on mutable Docker images and library versions. State files and metadata are not preserved, so rollback cannot be guaranteed. Identical code may yield divergent results under different environments, breaking referential consistency.

R4: Continuous Control Monitoring. The CI/CD workflow runs syntax checks but no policy validation. Separation of duties and multi-party approvals are not automated. Monitoring is reactive rather than preventative; control assurance cannot be measured continuously.

R5: End-to-End Evidence Chain. No unified record links input data, transformations, configurations, and outputs. Logs are scattered across components with no correlation or immutability. Lineage reconstruction requires manual collation, undermining the principles of referentiality and verifiability.

4.5 Summary

The baseline pipeline demonstrates a functioning but ungoverned ETL workflow. It moves data reliably from extraction to storage but depends on convention rather than control. Each tool—Airflow, PySpark, S3, Terraform, and GitHub Actions—fulfils its operational role but not its assurance role. None provide signed artifacts, immutable logs, or reproducible lineage. These gaps define the starting point for the DevSecOps architecture in Chapter 5, which integrates governance and verification to transform a working pipeline into an auditable one.

Chapter 5

Proposed Architecture

This chapter presents the proposed DevSecOps architecture for governing an ETL pipeline in a regulated environment. It builds directly on the baseline pipeline from Chapter 4, retaining the same essential data movement while adding the control, evidence, and verification mechanisms required to satisfy the governance objectives defined in Chapter 3.

The design should be read as a governance architecture rather than merely a runtime topology. Its purpose is not only to execute extraction, transformation, and loading tasks, but to ensure that each operational change, deployment, and dataset version can be explained after the fact using authoritative evidence. In this sense, the proposed contribution sits above the business logic of the pipeline and governs the conditions under which that logic is allowed to operate.

The architecture is described at escalating levels of detail. It begins with a lifecycle view of the governed system, then decomposes the major architectural subsystems, then narrows to the runtime data and metadata flows, and finally follows a representative run from change request to dataset version and audit reconstruction. Consistent with the methodology, the controls described here should be interpreted as representative minimum controls for the thesis prototype, not as a claim of production-complete assurance.

5.1 Lifecycle Overview

At the highest level, the proposed architecture links four concerns that are separate or weakly connected in the baseline: change intent, release control, runtime execution, and audit reconstruction. The central design claim is that a dataset should never appear as an isolated output. It should instead be the observable consequence of an approved change, an attested release, a specific infrastructure state, and a recorded execution context.

Figure 5.1 presents this lifecycle as a single governed process. The top lane captures change management, where a GitHub Issue establishes the initial statement of intent and a pull request subjects the change to automated and human

review. The second lane captures the execution plane, where an approved artifact is used to run the pipeline and produce a dataset version. The third lane introduces blocking controls, particularly artifact verification and runtime data quality gates. The fourth lane captures the evidence layer, where logs, run metadata, lineage metadata, and infrastructure references are retained so that an auditor can reconstruct what happened without manually correlating disconnected systems.

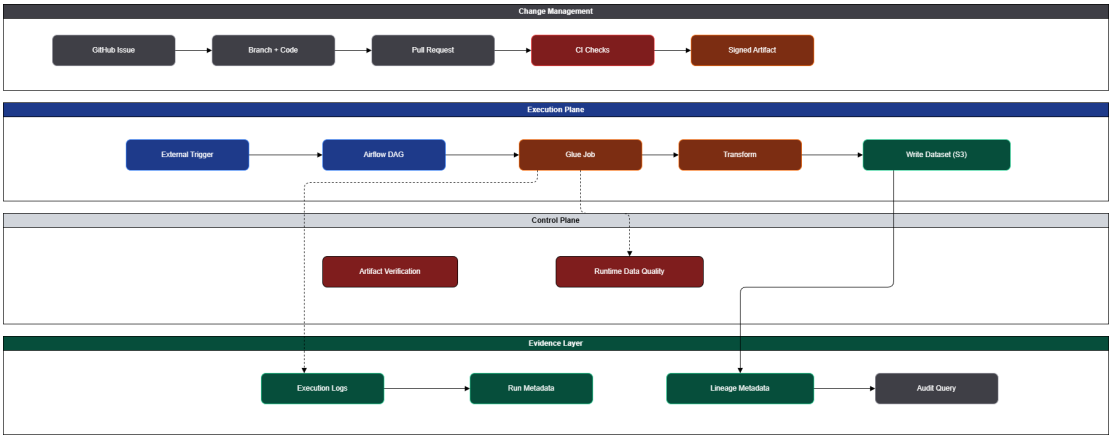

Figure 5.1: Unified lifecycle of the governed ETL architecture.

Change intent as the start of lineage. The evidence chain begins before the pipeline runs. A governed change must start with a documented issue or change request that records why the change is being made and what requirement or defect it addresses. This addresses Requirement R1 by preserving decision authority alongside code authorship.

Release control as the bridge between code and runtime. The architecture does not allow code to move directly from repository to execution environment by convention alone. It introduces CI/CD gates, policy evaluation, and artifact attestation so that runtime execution is tied to an approved release object rather than to an ambiguous repository state. This is the primary mechanism by which Requirements R2 and R4 are operationalised.

Runtime control as a governed execution path. The runtime path remains recognisably ETL: data are ingested, transformed, validated, and written to governed storage. What changes is that each step operates within a recorded control context. The run is identified, the deployed artifact is referenced, the infrastructure state is captured, and promotion to the final analytical layer is blocked unless quality conditions are met.

Audit reconstruction as an explicit design goal. The final concern is not throughput or convenience but reconstructability. Requirement R5 demands that

the system answer questions such as: Which change produced this dataset version? Which approved artifact ran? Which infrastructure state was active? Which control checks passed or failed? The architecture is therefore designed backwards from the audit query as much as it is designed forwards from the ETL process.

5.2 Structural Decomposition

Where the lifecycle view explains the system behaviourally, the high-level solution architecture explains it structurally. Figure 5.2 shows the major subsystems and boundaries that participate in the governed pipeline.

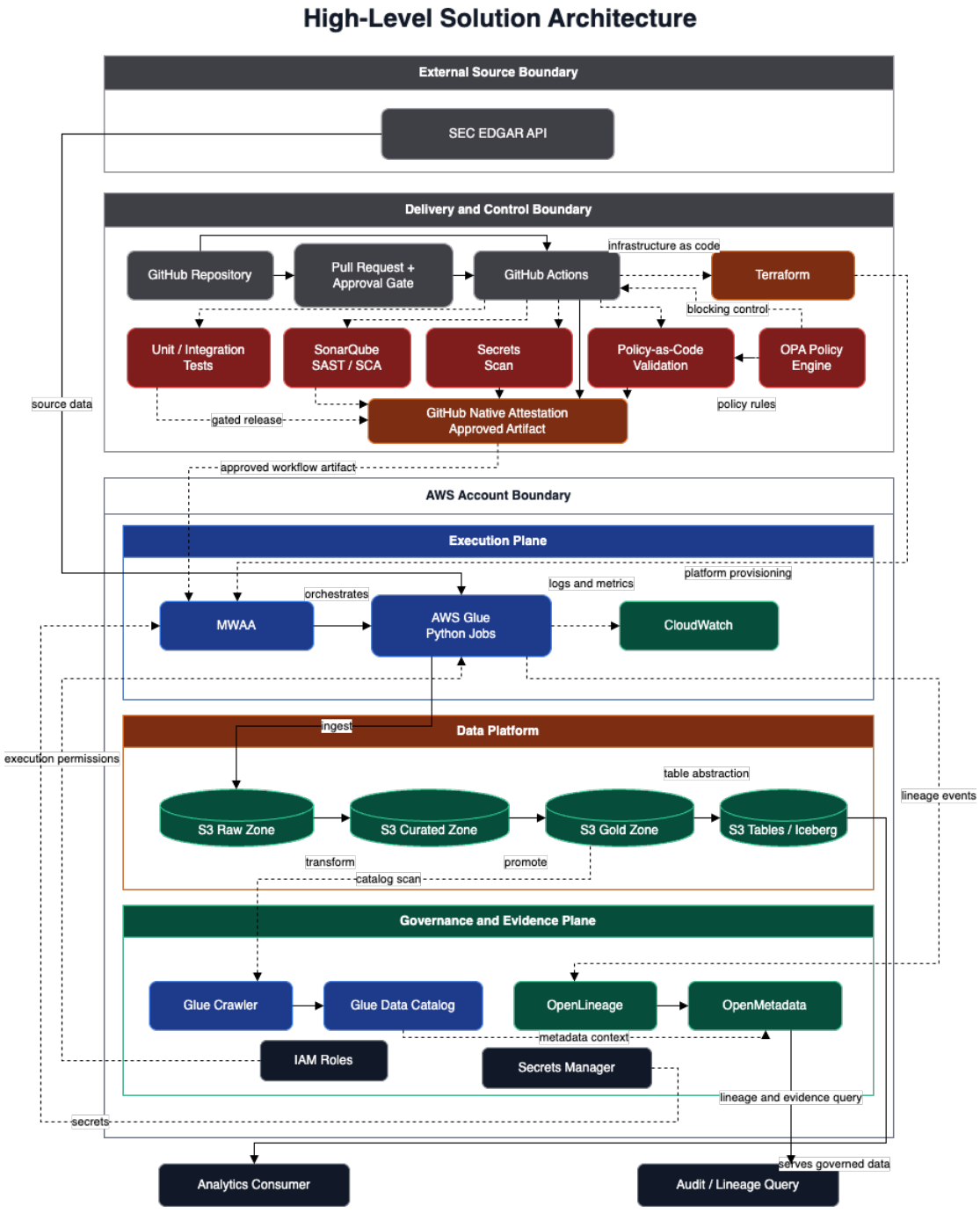

Figure 5.2: High-level solution architecture for the proposed pipeline.

External source boundary. The source system remains the SEC EDGAR API, preserving comparability with the baseline. This is important analytically: the proposed architecture is not an alternative business pipeline, but a governed form of the same core workload.

Delivery and control boundary. The first major subsystem is the delivery and control boundary. It combines the GitHub repository, pull request approval gate, GitHub Actions, Terraform, automated tests, security scanning, policy-as-code validation, and artifact attestation. This boundary is the control-plane entry point for all code and configuration change. In the implementation contract, it is also where the logical release manifest is assembled, linking the deployable Airflow and Glue assets to the CI run and attestation evidence that produced them.

AWS account boundary. Within the cloud target, the architecture is divided into three bands: *execution plane*, *data platform*, and *governance and evidence plane*. This separation matters because the thesis is evaluating more than runtime correctness. The execution plane runs the workflow, the data platform stores the outputs, and the governance plane preserves the context that makes those outputs auditable.

Execution plane. The execution plane consists of MWAA, AWS Glue Python jobs, and CloudWatch. MWAA acts as the orchestration surface for governed cloud execution, while Glue performs the operational work of ingestion, transformation, quality checking, and promotion. CloudWatch provides operational telemetry and supporting logs. During development, the same orchestration logic can be exercised locally in Docker-based Airflow, but the governed target architecture remains MWAA in the AWS environment.

Data platform. The data platform is intentionally layered. Raw extracted data are written to an S3 raw zone. Normalised and contract-coerced records are written to an S3 curated zone. Promoted analytical outputs are then written to the gold layer, represented conceptually as an S3 gold zone and technically as Amazon S3 Tables backed by Iceberg semantics. This progressive structure preserves the staging logic of ETL while distinguishing between transient intermediates and governed analytical state.

Governance and evidence plane. The governance and evidence plane contains Glue Crawler, Glue Data Catalog, OpenLineage, OpenMetadata, IAM Roles, and Secrets Manager. These components provide metadata context, service boundaries, secret handling, and queryable lineage. The design intent is that runtime metadata should not remain trapped inside operational logs. It should be surfaced into a metadata plane that can join lineage events, catalog state, change references, and artifact provenance.

Downstream consumption. The architecture ends with two downstream actors: the analytics consumer and the audit or lineage query user. This distinction is deliberate. The analytics consumer needs governed data products; the auditor

needs reconstructed evidence. A mature architecture must satisfy both, but they query different parts of the system for different reasons.

5.3 Governed Process Maps

The high-level structure becomes clearer when decomposed into the concrete processes that operate across it. The process maps in this section show how the same architecture behaves under different concerns: change governance, continuous delivery, infrastructure change, runtime execution, and audit reconstruction.

5.3.1 Change Management Workflow

Figure 5.3 shows how the architecture treats change management as the first control surface rather than as an informal project-management layer.

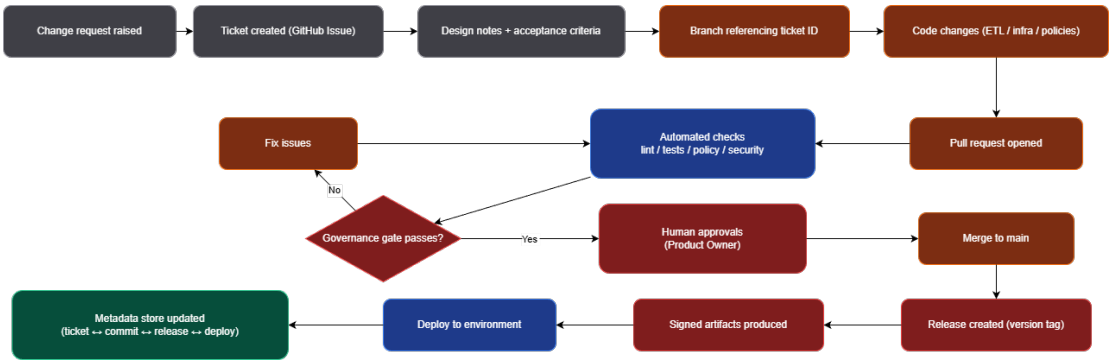

Figure 5.3: Governed change management workflow.

The process begins with a change request and a GitHub Issue containing design notes and acceptance criteria. Development then proceeds on a branch that references the ticket identifier, ensuring that the repository history carries forward the originating governance reference. A pull request triggers automated checks covering linting, tests, policy validation, security review, and ticket enforcement. If those checks fail, the change returns for remediation; if they pass, the change still requires explicit human approval from the Code Owner and Data Owner before merge.

This workflow operationalises Requirement R1 by binding intent, authorship, and approval into a single continuous chain. It also contributes to Requirement R4, because policy and validation are not optional review activities but blocking controls in the merge path.

5.3.2 CI/CD Release Workflow

Figure 5.4 narrows to the release path itself.

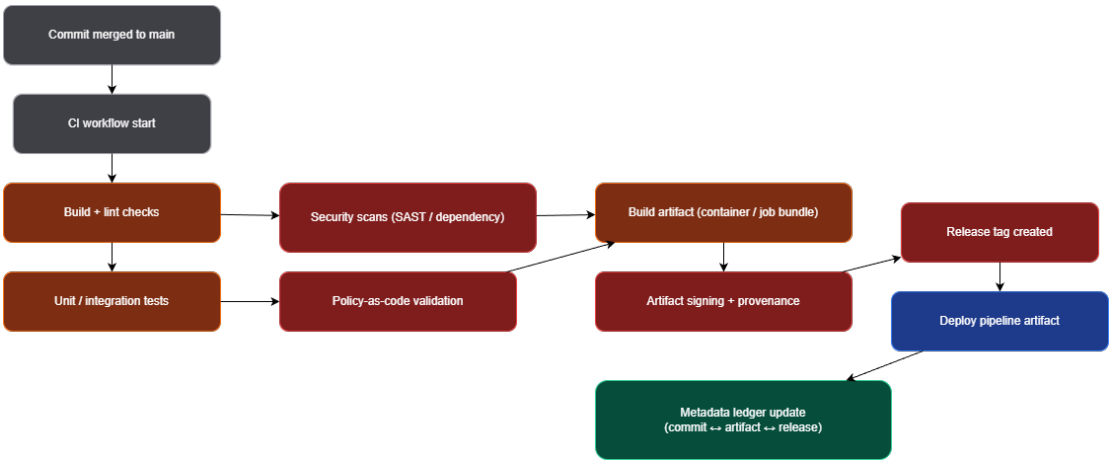

Figure 5.4: CI/CD workflow for packaging, attestation, and deployment.

Once a change is merged, GitHub Actions performs build and lint checks, unit and integration testing, security scanning, and policy-as-code validation. Only then is the deployment artifact built, attested, and released for deployment. This sequencing is critical. In the baseline, deployment is driven primarily by repository events; in the proposed architecture, deployment is driven by the successful creation of an approved release object. The conceptual trace becomes: `commit -> CI run -> approved artifact -> attestation -> release -> deployment`.

This is the principal mechanism for satisfying Requirement R2. Integrity is not assumed because a file exists in the repository; it is asserted only after the artifact has passed the required gates and has provenance attached to it.

5.3.3 Infrastructure Change Workflow

Figure 5.5 shows that infrastructure change is governed using the same logic as application change.

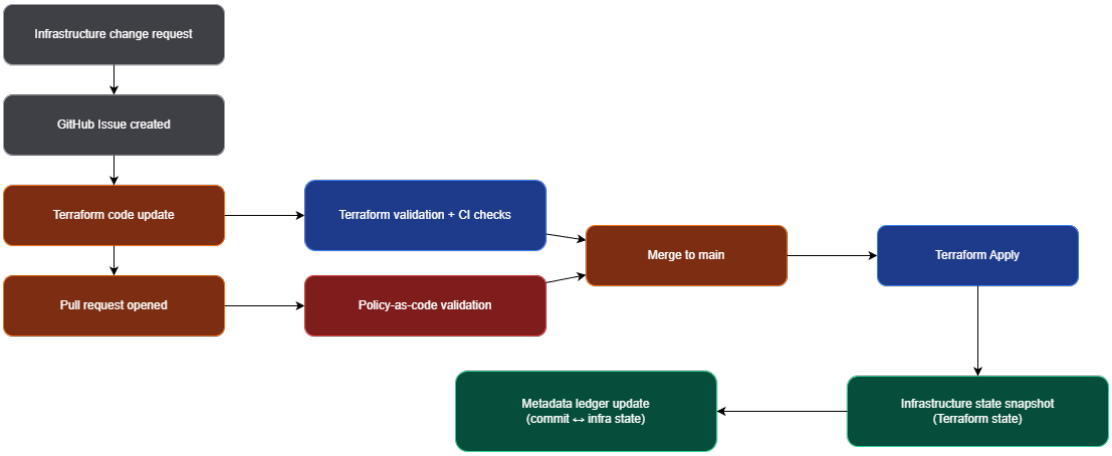

Figure 5.5: Infrastructure change workflow with Terraform state capture.

Terraform changes begin with an issue, pass through pull-request review, and are subjected to validation and policy checks before merge. After `terraform apply`, the applied infrastructure state version is recorded as evidence. This gives the architecture a durable reference to the platform configuration that existed at the time a dataset was produced. That reference is essential for Requirement R3 because reproducibility depends not only on code version, but also on the state of the environment in which the code ran.

5.3.4 Runtime ETL Execution Workflow

Figure 5.6 shows the governed runtime path.

Figure 5.6: Governed runtime execution workflow.

The flow begins with a pipeline trigger and Airflow DAG creation, followed by verification that the runtime is using an approved artifact. Glue jobs then read the source dataset, perform schema validation, run data quality and data contract checks, apply the transformation, and produce curated output. Only after the post-transform checks pass is the dataset promoted to the gold layer. Execution logs, run evidence, and lineage evidence are recorded as separate but linked outputs.

The important difference from the baseline is that quality and promotion are not descriptive afterthoughts. They are blocking decisions in the runtime path. This converts data quality from an operational best practice into a control instrument that directly governs whether an output is permitted to become a governed dataset version.

5.3.5 Lineage and Audit Reconstruction Workflow

Figure 5.7 reverses the direction of reasoning and begins with the audit question instead of the pipeline trigger.

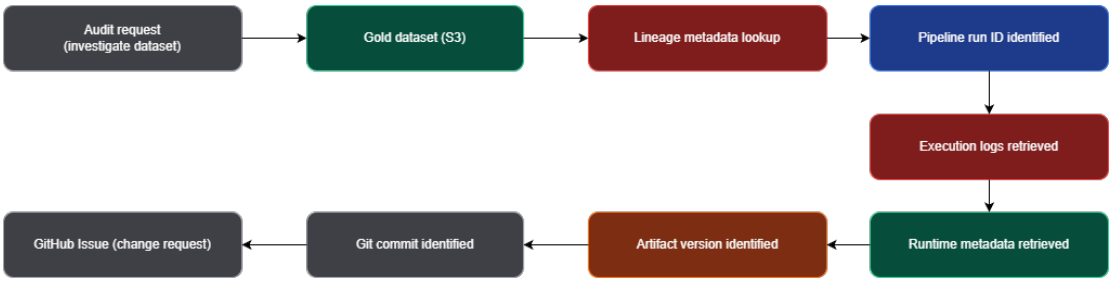

Figure 5.7: Audit reconstruction workflow from dataset to change intent.

An audit request begins with a dataset under investigation. From that gold dataset, the system performs a lineage lookup to identify the pipeline run ID, retrieves runtime metadata as authoritative evidence, consults execution logs as supporting evidence, and identifies the artifact version, Git commit, infrastructure state version, and originating change request. The intended result is a coherent reconstruction of `data + code + infrastructure + intent`.

This workflow directly expresses Requirement R5. A system that can only produce logs has not solved the governance problem; it has only moved the manual work to a later stage. The proposed design aims to make reconstruction a first-class system capability.

5.4 Data and Metadata Flows

The process maps describe behaviour over time. The data-flow diagrams explain how runtime assets, datasets, and evidence objects move between components. Three views are important: the overall system data flow, the governed runtime flow, and the metadata and evidence flow.

5.4.1 System Data Flow

Figure 5.8 presents the end-to-end system data flow across the repository, delivery controls, AWS runtime, metadata services, and downstream consumers.

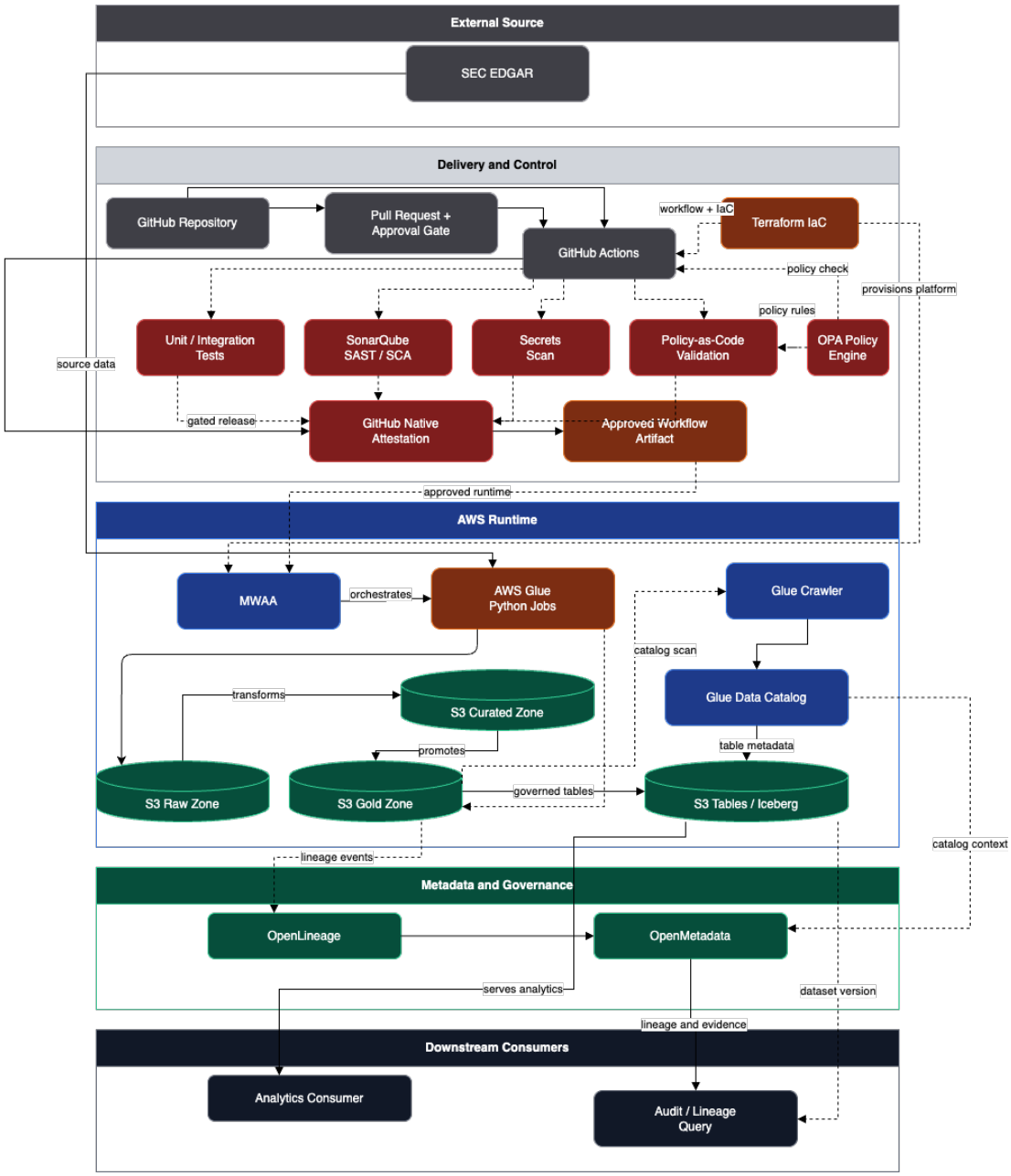

Figure 5.8: System-level data flow across delivery, runtime, and metadata layers.

The governing repository and CI/CD systems produce an approved workflow artifact. That artifact, together with Terraform-provisioned infrastructure, establishes the approved runtime context for MWAA and Glue. Runtime data then move from SEC EDGAR into the raw zone, from raw to curated, and from curated to the gold layer. The gold layer is surfaced as governed tables through S3 Tables/Iceberg, with Glue Crawler and Glue Data Catalog providing additional metadata context.

This system view shows why the proposed architecture is more than an orches-

tration pattern. The delivery path, runtime path, and metadata path are linked so that the data product and the control product remain synchronised.

5.4.2 Runtime Data Flow

Figure 5.9 narrows from the whole system to a single governed execution context.

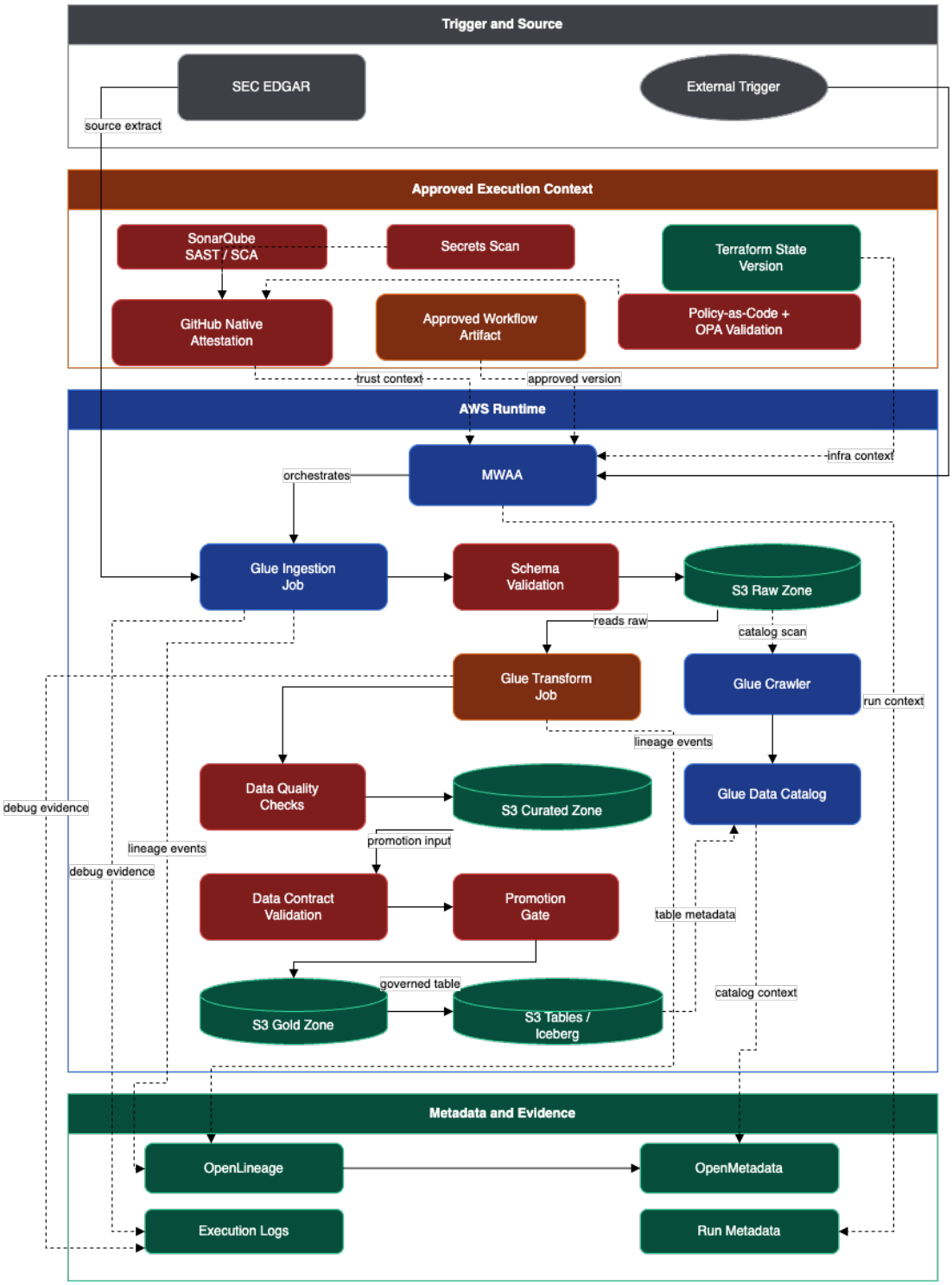

Figure 5.9: Runtime flow showing the approved execution context and control gates.

This view emphasises that runtime execution is dependent on prior control outcomes. External triggering alone is insufficient. The run must also inherit an

approved workflow artifact, its attestation, the relevant Terraform state version, and the evidence that required policy and security gates have already passed. Once in runtime, the flow proceeds through MWAA, Glue ingestion, schema validation, the raw zone, Glue transformation, the curated zone, data quality checks, data contract validation, the promotion gate, and finally the gold dataset and table abstraction.

The diagram therefore combines two forms of control. The first is pre-execution control, inherited from CI/CD. The second is in-execution control, applied to data and transformation results before promotion. Together they bind Requirements R2, R3, and R4 to the same execution path.

5.4.3 Metadata and Evidence Flow

Figure 5.10 describes the movement of evidence and references rather than business data.

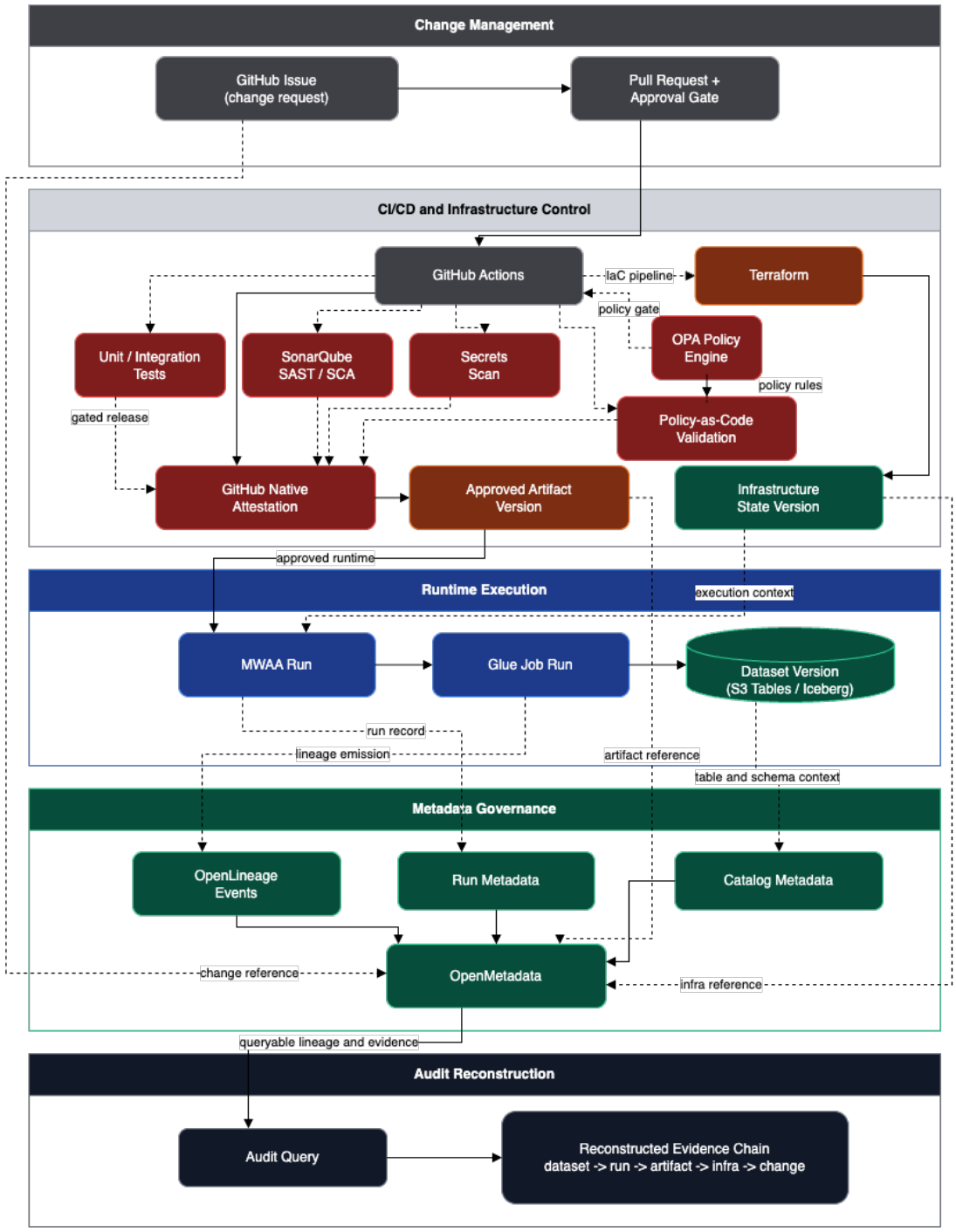

Figure 5.10: Metadata and evidence flow supporting audit reconstruction.

The metadata flow begins in change management with the issue and pull request, continues through GitHub Actions, OPA policy evaluation, attestation, approved artifact version, Terraform, and infrastructure state version, then enters the runtime as MWAA and Glue run metadata. At runtime, dataset versions, lineage events, run records, and catalog metadata are pushed into OpenMeta-

data, which acts as the query hub for reconstructed evidence chains. The intended audit path is explicit in the diagram: `dataset -> run -> artifact -> infrastructure -> change`.

This design choice is central to the thesis. Logs alone are retained only as supporting evidence. The authoritative audit chain is constructed from linked metadata references that can be queried consistently and validated independently.

5.5 Worked Example: One Governed EDGAR Run

To make the architecture concrete, this section follows the representative pipeline slice implemented in the prototype repository. The current codebase models a single governed EDGAR flow centred on the `edgar_governed_pipeline`, with separate ingestion, transformation, quality, and promotion steps plus a versioned run manifest.

1. Change and release preparation. A change begins as a GitHub Issue and is implemented on a branch that encodes the governing issue identifier. The pull request body carries phase and requirement references, which are parsed into machine-readable governance inputs for CI policy checks. Automated validation then evaluates branch naming, issue linkage, template completeness, tests, and policy conditions before the change is allowed to merge. Once approved, the release process produces a logical release object that binds together the workflow artifact, the CI run, the attestation record, and the deployment target.

2. Run initialisation. When the DAG is triggered, it first creates a run context and an initial manifest. In the current prototype, that manifest records the `run_id`, `pipeline_id`, `dataset_date`, `source_ref`, `release_manifest_ref`, `terraform_state_ref`, `change_ref`, the expected job references, the gold table coordinates, and the initial status. This occurs before any data movement, which means the execution is evidence-bearing from its first step rather than only at completion.

3. Ingestion. The ingestion job reads the EDGAR source payload, expands the `filings.recent` structure into row-wise records, normalises the key fields, and writes two raw outputs: the original payload and the normalised record set. In the current implementation these outputs are partitioned by `dataset_date` and `run_id`. The manifest is then updated with the raw output locations and the raw row count. At this point, the run can already prove which source was consulted and where the first persisted outputs were written.

4. Transformation and contract coercion. The transformation job reads the normalised raw records, enriches them with the run identifier and dataset date, coerces them to the curated data contract, and writes the curated dataset to the curated zone. The manifest is updated again with the curated output location, the contract reference, and the curated row count. This step is significant because it converts the raw landing zone into a schema-governed intermediate suitable for policy-based promotion.

5. Runtime quality gate. The data quality step evaluates the curated output against a small but explicit control set: the curated object must exist, it must match the declared contract, the row count must be greater than zero, and required identifiers must be populated. The result is recorded in the manifest as a structured `quality_result`. If any check fails, the run is finalised with a `failed_quality` status and no dataset version is emitted. This is the runtime expression of the blocking controls shown in the process maps.

6. Promotion and dataset versioning. If the quality gate passes, the promotion step writes the approved records to the gold table target. The current prototype computes a deterministic dataset version from the gold table identity and a digest of the promoted content, then stores both the dataset version and the write result in the manifest. This matters for Requirement R3: replayability and reconstruction are only credible if the final version identifier is stable for identical governed inputs and execution conditions.

7. Final manifest and audit lookup. At the end of the run, the manifest contains the raw outputs, curated outputs, quality result, row counts, dataset version, gold write result, timestamps, and final status. In the integration flow used by the prototype, a representative run is linked to references such as `release-manifests/dev-edgar-v1.js` `terraform-state/dev/serial-0001`, and `issue/EDGAR-1`. An auditor starting from the resulting dataset version can therefore traverse back to the run manifest, then to the release manifest, infrastructure state reference, and governing change request. The evidence chain is not inferred; it is recorded explicitly.

Worked-example significance. This example illustrates the architectural argument of the thesis. The novelty is not that EDGAR data can be ingested or transformed; the baseline already demonstrates that. The novelty is that the governed run emits enough linked context to explain why the dataset exists, under what authority it was produced, which control conditions were satisfied, and how the same state could be replayed or challenged later.

5.6 Summary

The proposed architecture transforms the baseline ETL pipeline into a governed data-delivery system. It introduces a control plane that begins with change intent, continues through automated and human release gates, governs runtime promotion decisions, and preserves a queryable evidence chain spanning code, configuration, data, and infrastructure.

At the conceptual level, the architecture integrates change management, continuous delivery, runtime orchestration, metadata governance, and audit reconstruction into one lifecycle. At the structural level, it separates the execution plane, data platform, and governance plane while linking them through attested artifacts, infrastructure references, lineage events, and run manifests. At the operational level, it culminates in a governed pipeline run whose dataset version can be reconstructed back to its originating change request. This architecture provides the design foundation for the prototype and the evaluation that follow.

Chapter 6

Results

This chapter presents the results from implementing and exercising the proposed governed ETL architecture. The results are intentionally scoped to the implemented architecture. They demonstrate whether the architecture can produce the forms of evidence required by the evaluation framework in Chapter 3. The formal assessment of those results against the requirements is provided in Chapter 7.

The implemented prototype uses the governed EDGAR pipeline described in Chapter 5. The pipeline ingests a representative SEC EDGAR fixture, writes raw and curated records through the S3 storage zones, validates the curated output against a data contract, promotes the passing dataset to the governed gold table layer, and records run evidence in JSON manifests and metadata artifacts. This setup was sufficient to test the central claim of the architecture: that a produced dataset version can be reconstructed back to the run, code bundle, release manifest, infrastructure reference, and change record that produced it.

6.1 Implementation Details

The implementation continues the architecture described in Chapter 5 by materialising the proposed controls across the target orchestration, infrastructure, metadata, and evidence planes. The pipeline was implemented as a governed Airflow workflow intended for MWAA, with Terraform defining the AWS resources, S3 separating the raw, curated, manifest, and gold data zones, and the metadata layer recording lineage, quality, release, and reconstruction evidence. The important implementation result is that each produced dataset version carries enough references to connect the data product back to the release manifest, change record, code bundle, infrastructure state, quality result, OpenLineage event, and OpenMetadata records.

The target implementation therefore follows the same split used in the architecture chapter. The orchestration plane schedules and executes the EDGAR workflow through Airflow on MWAA. The data plane stores source, intermedi-

ate, manifest, and promoted outputs in the S3-backed zones. The governance plane uses GitHub Actions, OPA policy checks, release manifests, and artifact attestation to control change. The evidence plane then persists the run manifest, audit-chain file, lineage event, catalogue record, and schema contract so that the produced gold dataset can be reconstructed without relying on memory of the deployment.

The main implementation hurdle was the cost of repeatedly provisioning the managed orchestration environment. The MWAA environment took approximately 45 minutes to create from Terraform and approximately 15 minutes to tear down. That made full rebuilds too slow for normal development, so a Docker-based Airflow environment was used during local development to allow faster iteration before deployment to the target architecture.

The second major issue during development was the recent instability of GitHub. Frequent outages affected the ability to develop and test the system, with particular frustration caused by the GitHub merge-queue incident. Because the target architecture depends on GitHub for issues, pull requests, release controls, ownership, and attestation, these incidents reinforced the need to retain independent release manifests, hashes, and run manifests rather than relying only on platform state. These issues create uncertainty in a system where the foundational principle is trust. Questions about reliance on GitHub are therefore returned to later in the thesis.

The detailed implementation version and platform reference tables are provided in Appendix A so that the results discussion can focus on the evidence produced by the architecture rather than the support stack inventory.

6.2 Observed Results

The results in this chapter are presented as a worked example following one representative governed EDGAR run. The purpose of doing this is to show how the architecture behaves when an auditor starts from a produced dataset and follows the evidence chain backward. The example run ingested the representative SEC EDGAR fixture, transformed it into the curated contract shape, executed the quality gate, promoted the passing output to the gold table, and wrote the surrounding run, lineage, catalogue, and audit-chain evidence.

For this demonstration run, the promoted gold dataset version was `9b5fd26a2005c02b`. The value is used throughout the remainder of this chapter as the example lookup key, not as a claim that the identifier itself is important outside this run. Repeated execution over the same governed input, contract, and transformation logic resolved to the same version identifier, which is the expected behaviour because the version is derived from promoted content and table identity. The important result was therefore not the volume of data processed, but that the run produced a stable governed output with enough surrounding evidence to explain that output without manual log collation.

Table 6.1 summarises how the artifacts in this single worked example point to each other. The table should be read as the traceback path for the demonstration run rather than as a general inventory of all possible evidence files.

Table 6.1: Evidence artifact linkage used for audit reconstruction.

Artifact	**Pointer field**	**What it resolves to**
Gold dataset	`dataset_version`	Resolves to the audit-chain artifact for `9b5fd26a2005c02b`.
Audit-chain artifact	`manifest_ref`	Resolves to the run manifest that produced the dataset.
Run manifest	`release_manifest_ref`	Resolves to the release manifest that defines the DAG and job files.
Run manifest	`change_ref`	Resolves to the governed change record, including the `EDGAR-1` implementation record.
Run manifest	`terraform_state_ref`	Resolves to the infrastructure state reference for the execution environment.
Run manifest	`curated_outputs.contract_path`	Resolves to the curated contract used by the quality gate.
Run manifest	`code_bundle.artifacts`	Resolves to hashed DAG, ingest, transform, quality, and promotion files.
Run manifest	`metadata_refs`	Resolves to OpenLineage, OpenMetadata, and audit-chain metadata records.
OpenLineage event	`inputs` and `outputs`	Resolves source, raw, curated, manifest, and gold-table lineage endpoints.
OpenMetadata dataset record	`upstreams` and `manifest_ref`	Resolves the dataset back to upstream inputs and the producing run manifest.

In this demonstration run, the quality gate showed that the curated dataset existed, matched the declared contract, contained non-empty records, and had required identifiers populated. These are deliberately simple controls, but they demonstrate the runtime enforcement pattern: a dataset is not promoted because a task finishes; it is promoted because the required checks pass and their results are recorded.

The generated run manifest is the main evidence object for the example. For the successful run, it records the run identifier, source reference, raw outputs,

curated output, contract path, row counts, quality result, gold table coordinates, dataset version, release manifest reference, Terraform state reference, and change reference. In the complete governed Airflow run, the manifest also records source-control provenance, code bundle hashes, attestation fields, reference checks, OpenLineage output, OpenMetadata-style records, and the audit-chain path. This satisfies the practical requirement that the output can be explained without manually reconstructing the run from unrelated logs.

6.3 Evidence Schemas

The results are grounded in the structure of the evidence artifacts rather than only in task success. Each artifact below is materialised as JSON and records a different part of the governed chain. In the worked example, these schemas explain what each evidence artifact is expected to carry as the run moves from ingestion through promotion and then into audit reconstruction.

Table 6.2: Run manifest schema.

Field	Explanation
`run_id`	Identifies the specific governed execution.
`pipeline_id`	Identifies the pipeline definition that ran.
`dataset_date`	Identifies the dataset partition being produced.
`source_ref`	Records the upstream source object or fixture used for ingestion.
`release_manifest_ref`	Links execution to the approved release definition.
`terraform_state_ref`	Links execution to the infrastructure state reference.
`change_ref`	Links execution to the governing change record.
`job_refs`	Lists the ingest, transform, quality, and promotion jobs used by the run.
`raw_outputs`	Records raw payload and normalized raw output locations.
`curated_outputs`	Records curated output and contract path.
`gold_write_result`	Records promoted gold table write details.
`row_counts`	Stores raw, curated, and promoted count consistency evidence.
`quality_result`	Captures the blocking quality-gate result and per-check outcomes.
`dataset_version`	Identifies the promoted gold dataset version.
`reference_checks`	Records whether release, infrastructure, and change references resolved at runtime.
`source_control`	Captures repository, branch, commit, and commit URL evidence.
`code_bundle`	Captures release-manifest and job artifact hashes.
`attestation`	Captures GitHub attestation provider and subject fields.
`metadata_refs`	Links to catalogue, lineage, and audit-chain artifacts.
`status`	Records the final run state.
`started_at`	Records when the manifest began tracking the run.
`completed_at`	Records when the run evidence was finalised.

Table 6.3: Release manifest schema.

Field	Explanation
`release_id`	Names the governed release, such as `dev-edgar-v1`.
`pipeline_id`	Binds the release to `edgar_governed_pipeline`.
`dag_bundle_ref`	Points to the Airflow DAG bundle included in the release.
`jobs`	Lists the ingest, transform, quality, and promotion job files.
`source_control`	Records commit, branch, repository, and commit URL when provided by the release workflow.
`artifact_bundle`	Reserves the package path, digest, subject digest, and attestation bundle path.
`attestation`	Records GitHub attestation provider, run, workflow, URL, and subject digest fields.
`notes`	Documents the release purpose or environment.

Table 6.4: Change record schema.

Field	Explanation
`change_id`	Names the governing change, for example `EDGAR-1`.
`title`	Describes the implementation or rehearsal governed by the change.
`source`	Identifies the governance source used for the change record.
`source_control`	Stores commit, branch, repository, and commit URL fields when available.
`attestation`	Mirrors the GitHub attestation fields used by release and run evidence.

Table 6.5: Curated contract schema.

Field	Explanation
`dataset`	Names the contract target, `edgar_curated`.
`description`	Explains the deterministic governed EDGAR filing slice.
`required_fields`	Requires `accession_number` and `company_cik`.
`columns`	Defines 8 expected curated columns and their string types.

Table 6.6: Audit-chain schema.

Field	Explanation
`dataset_version`	Starts reconstruction from the promoted gold dataset version.
`gold_table_uri`	Identifies the promoted gold table location.
`pipeline_id`	Identifies the pipeline that produced the dataset.
`run_id`	Identifies the producing run.
`manifest_ref`	Resolves the dataset to the producing run manifest.
`source_ref`	Identifies the source input used by the run.
`release_manifest_ref`	Links the dataset to release evidence.
`terraform_state_ref`	Links the dataset to infrastructure evidence.
`change_ref`	Links the dataset to change evidence.
`source_control`	Records commit provenance.
`code_bundle`	Records release hashes and job hashes.
`attestation`	Records attestation provider and subject fields.
`quality_status`	Records whether the dataset passed the quality gate before promotion.

Table 6.7: OpenLineage event schema.

Field	Explanation
`eventType`	Records the lineage event type.
`eventTime`	Records the lineage event time.
`job`	Identifies the pipeline namespace and job name.
`run`	Identifies the OpenLineage run UUID and governance facets.
`inputs`	Lists source, raw, and curated upstream datasets.
`outputs`	Lists the run manifest and promoted S3 Tables dataset.
`facets.version`	Stores dataset version, content digest, and promoted records path.
`producer`	Records the producing metadata component.
`schemaURL`	Records the OpenLineage schema reference.

Table 6.8: OpenMetadata run-record schema.

Field	Explanation
`entity_type`	Marks the record as a pipeline run.
`pipeline_id`	Identifies the producing pipeline.
`run_id`	Identifies the producing run.
`dataset_date`	Identifies the dataset partition.
`status`	Records run status.
`quality_status`	Records quality-gate status.
`row_counts`	Records count consistency.
`manifest_ref`	Links the record back to the run manifest.
`source_ref`	Links the record to the source input.
`release_manifest_ref`	Preserves the release reference.
`terraform_state_ref`	Preserves the infrastructure reference.
`change_ref`	Preserves the change reference.
`source_control`	Carries commit provenance into catalogue metadata.
`code_bundle`	Carries artifact hashes into catalogue metadata.
`attestation`	Carries attestation fields into catalogue metadata.

Table 6.9: OpenMetadata dataset-record schema.

Field	Explanation
`entity_type`	Marks the record as a dataset.
`dataset_version`	Identifies the promoted dataset version.
`content_digest`	Fingerprints the promoted dataset content.
`gold_table_bucket`	Identifies the gold table bucket.
`gold_namespace`	Identifies the gold table namespace.
`gold_table`	Identifies the gold table name.
`table_uri`	Locates the promoted gold table.
`records_path`	Locates the materialised records file.
`write_result_path`	Locates the write-result evidence file.
`row_count`	Records count evidence.
`run_id`	Links the dataset back to the producing run.
`manifest_ref`	Links the dataset back to the producing run manifest.
`source_curated_uri`	Records the curated source used for promotion.
`upstreams`	Preserves upstream lineage inputs.
`source_control`	Carries code provenance into the dataset record.
`attestation`	Carries attestation fields into the dataset record.

6.4 Audit Reconstruction Result

The strongest result from the worked example is the audit reconstruction path. Starting only from the gold dataset version `9b5fd26a2005c02b`, an auditor can resolve the audit-chain artifact at `metadata/audit-chain/9b5fd26a2005c02b.json`. Within this example, that artifact identifies the pipeline `edgar_governed_pipeline`, the Airflow run that produced the output, the source fixture, the gold table URI `s3tables://gold/edgar/filings`, the release manifest reference, the Terraform state reference, the source-control commit, and the code bundle hashes for the DAG and job scripts.

The reconstruction then moves from the audit-chain artifact to the run manifest. The manifest records the raw payload and normalized raw record locations, the curated output location, the contract path, the quality result, and the promoted gold write result. The promoted write result records the table URI, dataset date, content digest, source curated URI, write-result path, and dataset version. This means the output can be traced backward from the gold table to the curated file, and then backward again to the raw source payload.

The same manifest also reconstructs the control path for the example run. The `reference_checks` block records that `release-manifests/dev-edgar-v1.json`, `terraform-state/aws-target.json`, and `changes/EDGAR-1.json` resolved suc-

cessfully. The `code_bundle` block records the release-manifest SHA-256 hash and individual hashes for `pipelines/edgar_governed_pipeline.py`, `jobs/ingest/edgar_ingest.py`, `jobs/transform/edgar_transform.py`, `jobs/quality/curated_quality_gate.py`, and `jobs/promote/promote_to_s3_table.py`. These hashes are important because they let the reconstruction identify the code that was intended to run, not only the path names of the files.

The schema evidence is also part of the reconstruction. The curated output points to `contracts/edgar_curated_contract.json`, which declares the expected columns and requires `accession_number` and `company_cik`. The quality result then records that the curated file matched this contract and that required identifiers were present. In this sense, the schema is not separate documentation. It is executable evidence used by the quality gate and retained in the manifest.

Finally, metadata publication connects the example run evidence file to the broader catalogue and lineage model. The `metadata_refs` block points to the OpenLineage completion event, the OpenMetadata-compatible run record, the OpenMetadata-compatible dataset record, and the audit-chain path. It also records the pipeline FQN, storage service name, raw, curated, and gold container FQNs, and lineage edge count. The worked example therefore has two directions of proof: the manifest can reconstruct the run from the dataset, and the metadata records can expose the same run through lineage and catalogue surfaces.

This result directly addresses the weakness observed in the baseline pipeline. In the baseline, a completed dataset could be associated with logs and output paths, but there was no single evidence object that bound the output to its change intent, runtime code, infrastructure reference, and quality decision. In the proposed implementation, those references are persisted together. The result is not merely more logging; it is a structured reconstruction path from dataset version back to operational authority.

6.5 Summary

The results show that the proposed architecture can produce a working evidence chain for a governed ETL run. In the representative worked example, the prototype generated a structured run manifest, 4 quality checks, 5 hashed code artifacts, 3 classes of metadata evidence, 6 lineage edges, and a stable dataset version that links the gold dataset back to code, infrastructure, and change references.

These results are reported from the representative EDGAR evaluation run and the development harness used to exercise the target design. That scope is intentional because the purpose of the results is to test the evidence chain from release to runtime output. The prototype demonstrates the central architectural claim: auditability improves when governance evidence is captured as part of the delivery and runtime workflow, rather than reconstructed manually after a dataset has already been produced.

Chapter 7

Evaluation

This chapter evaluates the results reported in Chapter 6 against the requirements defined in Chapter 3. The results chapter shows what the implemented architecture produced. This chapter considers whether those outputs satisfy the requirements and whether they satisfy the auditability definition used throughout the thesis.

The evaluation uses the definition from Chapter 2, where auditability is framed as the designed capability of a system to allow its processes, controls, and outcomes to be observed and verified by stakeholders. In that definition, auditability is not a single log file or a general claim that monitoring exists. It is made up of normativity, accountability, referentiality infrastructure, reliability, and verifiability [40]. The evaluation therefore asks whether the implemented fields and artifacts make those dimensions visible in the system.

7.1 Evaluation Approach

The assessment follows the R1–R5 requirement matrix from Chapter 3 and the control matrix in the proposed-solution repository. Each requirement is evaluated in three steps. First, the requirement is mapped back to the relevant auditability dimensions. Second, the concrete evidence fields are identified. Third, the observed result is judged against the requirement.

This approach deliberately treats the evidence artifacts as the primary object of evaluation. A pipeline run is not considered auditable only because it completed successfully. It is considered auditable if the produced dataset can be linked to the governing change, release, infrastructure reference, runtime code, quality decision, lineage event, and metadata record without manual log collation.

7.2 R1 Governance Accountability

R1 requires production-relevant changes to be linked to documented authority. In Chapter 3 this requirement was tied to normativity and accountability. Normativ-

ity is satisfied when the system records which governance rule or change process applies to an action. Accountability is satisfied when the run can be linked back to the actor, change, or approved release that authorised the action.

Table 7.1: R1 governance accountability evidence fields.

Field or artifact	Evaluation role
`change_ref`	Links the run to the governing change record.
`release_manifest_ref`	Links the run to the approved release definition.
`terraform_state_ref`	Links the run to the infrastructure state reference.
`reference_checks.change_ref`	Records whether the change record resolved at runtime.
`reference_checks.release_manifest_ref`	Records whether the release manifest resolved at runtime.
`reference_checks.terraform_state_ref`	Records whether the infrastructure reference resolved at runtime.
`source_control.commit`	Links the release and run evidence to the source revision.
Change record `change_id`	Names the governed change, for example `EDGAR-1`.
Release manifest `release_id`	Names the governed release, for example `dev-edgar-v1`.

The requirement is demonstrated because the implemented run is not recorded as an isolated Airflow execution. The run manifest contains the change, release, and Terraform references, and the reference-check block records whether each reference existed when the run evidence was produced. This means the run can be read as an authorised operational event rather than only as a scheduled task.

The link to the auditability definition is direct. The `change_ref` and `release_manifest_ref` fields provide normativity because they record the governing process that applied to the run. The source-control and release fields provide accountability because they bind the run to a concrete change record and code revision. The reference checks are important because they prevent the evidence chain from merely naming authority; they record that the named authority resolved.

7.3 R2 Information Security Assurance

R2 requires deployable artifacts and configuration to be verified for integrity and security before they are used. In Chapter 3 this was tied to reliability and verifiability. Reliability is concerned with whether the evidence faithfully represents the artifact that ran. Verifiability is concerned with whether an independent reviewer can test the claim.

Table 7.2: R2 information security assurance evidence fields.

Field or artifact	Evaluation role
`code_bundle.release_manifest_sha256`	Records the digest of the release manifest.
`code_bundle.artifacts.path`	Names each DAG or job artifact included in the bundle.
`code_bundle.artifacts.sha256`	Records a SHA-256 hash for each code artifact.
`source_control.repository`	Records the source repository used for the release.
`source_control.commit`	Records the source revision associated with the run.
`attestation.provider`	Records the attestation provider expected by the evidence model.
`attestation.subject_digest`	Carries the subject digest used to bind provenance to the release artifact.
GitHub Actions controls	Provide the CI/CD surface for linting, tests, policy checks, artifact upload, and attestation.

The requirement is demonstrated by the way the implementation records artifact identity. The code bundle includes hashes for the DAG and the ingest, transform, quality, and promotion jobs. These hashes make the run evidence more reliable because an auditor can compare the recorded digest with the file that is claimed to have run. The source-control fields then bind those artifacts to the repository and commit.

The release provenance fields extend the same chain to the release context. The evidence model records the provider, workflow, run, URL, and subject digest fields needed to connect the release bundle to GitHub-native provenance. In the implemented results, this requirement is evaluated through the demonstrated source revision, run reference, release reference, and SHA-256 artifact hashes.

This satisfies the auditability definition because the security claim is independently testable. An evaluator does not need to trust a written statement that the correct files were used. The evaluator can inspect `code_bundle.artifacts.sha256`, compare it with the artifact contents, and then follow `source_control.commit` back to the release source.

7.4 R3 Operational Risk Control

R3 requires the workflow to support reproducibility, recovery, and visibility over operational state. In Chapter 3 this requirement was tied to referentiality and reliability. Referentiality is satisfied when the run record connects the output to its source, intermediate, and runtime records. Reliability is satisfied when the recorded state is complete enough to support replay and diagnosis.

Table 7.3: R3 operational risk control evidence fields.

Field or artifact	Evaluation role
`run_id`	Identifies the specific governed execution.
`pipeline_id`	Identifies the pipeline definition that ran.
`dataset_date`	Identifies the dataset partition or business date.
`source_ref`	Records the source input used by the run.
`raw_outputs`	Records the raw payload and normalised raw output references.
`curated_outputs.curated_uri`	Records the curated dataset produced by transformation.
`quality_result.status`	Records whether the quality gate passed.
`gold_write_result`	Records the promoted gold table write result.
`dataset_version`	Records the stable output version used for reconstruction.
`status`	Records the final run outcome.
`completed_at`	Records when the run evidence was finalised.

The requirement is demonstrated because the run manifest records enough state to reconstruct how the output was created. It contains source, raw, curated, quality, promotion, and dataset-version fields in a single evidence object. This reduces operational risk because recovery no longer depends on reading unrelated logs and guessing which task output should be trusted.

The manifest validator strengthens the result. For a successful run, the validator requires a `dataset_version`, a `gold_write_result`, and metadata references to the OpenMetadata run record, OpenMetadata dataset record, OpenLineage event, and audit-chain path. It also requires reference checks for the release, Terraform, and change references. A run is therefore not merely successful because the final task completed. It is successful because the evidence needed for reconstruction is present.

This maps to referentiality because each stage of the data path points to the next stage. It maps to reliability because the system records the final status, quality result, row-count evidence, and completion time together with the output version. The result is a recovery-oriented run record rather than a best-effort operational log.

7.5 R4 Continuous Control Monitoring

R4 requires controls to run continuously and to produce evidence of their outcomes. In Chapter 3 this requirement was tied to normativity and accountability. Normativity is present when controls are embedded into the workflow rather than

applied after the fact. Accountability is present when the result of each control decision is retained.

Table 7.4: R4 continuous control monitoring evidence fields.

Field or artifact	Evaluation role
`quality_result.checks.curated_output_exists`	Records whether the curated output existed.
`quality_result.checks.schema_matches_contract`	Records whether the curated output matched the contract.
`quality_result.checks.row_count_gt_zero`	Records whether the curated output contained records.
`quality_result.checks.required_columns_non_null`	Records whether required identifiers were populated.
`curated_outputs.contract_path`	Links the quality decision to the data contract.
`reference_checks`	Records whether governance references resolved before the run evidence was accepted.
`metadata_refs.openlineage_events_path`	Records where the lineage event evidence was written.
`metadata_refs.openmetadata_records_path`	Records where the run metadata evidence was written.

The requirement is demonstrated because the implementation converts controls into runtime evidence. The quality result is not a general pass/fail statement; it records named checks. The contract path also remains in the manifest, which means the schema decision can be traced back to the exact contract used for the check.

This matters for auditability because continuous monitoring must be observable. If a quality or governance check only appears in a log line, the control is difficult to test independently. In this implementation, the check result is a field in the manifest and therefore becomes part of the same evidence chain as the dataset version. The same pattern is used for governance references: `reference_checks` records whether the release manifest, Terraform state reference, and change record resolved.

R4 is therefore satisfied within the implemented scope. The system demonstrates that control execution and control outcome can be captured as structured runtime evidence. The remaining production concern is scale and policy coverage: more policies would be needed for a complete enterprise deployment, but the architecture pattern is demonstrated.

7.6 R5 End-to-End Evidence Chain

R5 is the strongest auditability requirement because it asks whether a promoted dataset can be reconstructed from evidence without manual log collation. In

Chapter 3 this requirement was tied to referentiality and verifiability. It is also the requirement that most directly tests the thesis claim.

Table 7.5: R5 end-to-end evidence chain fields.

Field or artifact	Evaluation role
`dataset_version`	Starting point for reconstruction from the gold dataset.
Audit-chain `manifest_ref`	Resolves the dataset version to the run manifest.
Audit-chain `release_manifest_ref`	Resolves the dataset version to the release manifest.
Audit-chain `terraform_state_ref`	Resolves the dataset version to the infrastructure state reference.
Audit-chain `change_ref`	Resolves the dataset version to the governing change.
Audit-chain `code_bundle`	Carries the runtime code hashes into the reconstruction path.
`metadata_refs.audit_chain_path`	Records where the audit-chain artifact was written.
OpenLineage `inputs`	Records source, raw, curated, and manifest lineage inputs.
OpenLineage `outputs`	Records the gold-table lineage output and dataset version facet.
OpenMetadata dataset `manifest_ref`	Links the catalogue dataset record back to the run manifest.

The implemented result satisfies R5 because the dataset version `9b5fd26a2005c02b` can be used as the first lookup key. From there, the audit-chain artifact points to the run manifest. The run manifest then points to the release manifest, Terraform state reference, change record, code bundle, quality result, OpenLineage event, and OpenMetadata records.

This is the clearest link back to the auditability definition. Referentiality is demonstrated because the evidence chain preserves the points of recording for source data, raw output, curated output, manifest storage, metadata publication, and gold promotion. Verifiability is demonstrated because those links are materialised as fields and paths rather than implied by naming convention. An evaluator can follow the fields directly: `dataset_version` to audit-chain file, audit-chain `manifest_ref` to run manifest, run manifest to governance references, and metadata references to lineage and catalogue records.

R5 is therefore demonstrated because the system provides a structured reconstruction path. The baseline pipeline could show that tasks ran and files existed, but it did not bind the output to the change, release, infrastructure, code, quality, lineage, and catalogue evidence in one chain. The proposed architecture does.

7.7 Evaluation Summary

The evaluation shows that the proposed architecture satisfies R1–R5 within the implemented scope. More importantly, each requirement is satisfied in terms of the auditability definition from Chapter 2. R1 and R4 demonstrate normativity and accountability by recording the governing change, release, reference checks, and control outcomes. R2 and R3 demonstrate reliability by recording artifact hashes, source-control provenance, quality status, and complete run state. R3 and R5 demonstrate referentiality by linking source, raw, curated, manifest, metadata, and gold records. R2 and R5 demonstrate verifiability because the recorded hashes, manifests, metadata references, and audit-chain fields can be inspected independently.

The result is that auditability is implemented as a set of linked evidence fields, not as a retrospective explanation. The proposed architecture therefore meets the thesis objective: it improves ETL governance by making the authority, integrity, runtime state, control decisions, and output reconstruction path part of the delivery and execution workflow.

Chapter 8

Discussion

The results and evaluation show that the proposed architecture can turn ETL governance into a set of linked evidence artifacts. The main finding is not that the EDGAR pipeline processed a large amount of data. The stronger finding is that a produced dataset version can be connected back to the run manifest, release manifest, infrastructure reference, change record, code bundle, quality decision, OpenLineage event, and OpenMetadata-compatible records. This changes the audit problem from a manual search across tools into a structured reconstruction exercise.

8.1 Interpretation of the Findings

The prototype supports the central argument of the thesis: auditability is stronger when evidence is captured during delivery and runtime rather than assembled after the fact. In the baseline pipeline, the system could show that tasks had run and that outputs existed, but the evidence was distributed across logs, storage paths, infrastructure state, and repository history. The proposed architecture instead treats those references as part of the data product's evidence model.

This is important because regulated data pipelines are not only judged by whether data appears in the correct destination. They are also judged by whether the organisation can explain who authorised the change, which code ran, which infrastructure state supported the run, what controls passed, and how the output can be reproduced or challenged. The worked example in Chapter 6 shows that these questions can be answered from the manifest and metadata chain, rather than from an informal investigation.

8.2 Reliance on GitHub

The most significant architectural tension is the reliance on GitHub. In the implemented design, GitHub provides issue tracking, pull request review, source-control history, CI/CD execution, release control, ownership signals, and attestation. This

is convenient because it places many governance functions close to the code. It also creates a concentration of trust. If GitHub is unavailable, degraded, or internally inconsistent, several parts of the control plane are affected at the same time.

The GitHub outages encountered during development made this issue more concrete. The problem is not simply that an outage slows down development. The deeper problem is that the architecture uses GitHub as both a workflow engine and a source of governance evidence. A platform incident can therefore affect the ability to approve changes, run controls, produce attestations, and inspect historical state. In a system whose purpose is to improve trust, this dependency needs to be treated as a design risk rather than an operational inconvenience.

The implementation mitigates this by storing independent release manifests, run manifests, hashes, audit-chain files, and metadata references. Those artifacts mean the produced evidence is not held only in GitHub's user interface or workflow history. The remaining design concern is that GitHub still remains the place where many approvals, workflow decisions, and release provenance references originate.

A stronger production architecture would split these responsibilities across multiple services. Source control and pull requests could remain in GitHub, but release authority could be backed by an independent artifact registry and signed release store. Attestation could be verified through a separate provenance service. Policy decisions could be recorded in a policy engine or governance evidence store rather than only in CI logs. Runtime manifests could remain in versioned object storage, and catalogue records could remain in a metadata system such as OpenMetadata. This would reduce the blast radius of a single platform incident and make the evidence chain more defensible.

8.3 Implications for Practice

The practical implication is that DevSecOps controls should be designed as evidence-producing controls, not only as blocking gates. A failed check can prevent a deployment, but an auditable system also needs to record which check ran, what it checked, what artifact it applied to, and how that decision links to the final data product. The proposed architecture demonstrates this pattern through reference checks, quality results, code hashes, source-control fields, and metadata references.

The second implication is that data governance needs to be attached to runtime state. A release manifest is useful, but it is not sufficient unless the run manifest records that the release was actually used. A data contract is useful, but it is not sufficient unless the quality result records that the contract was checked before promotion. A lineage system is useful, but it is not sufficient unless the lineage event links to the same dataset version and run manifest as the rest of the evidence chain.

8.4 Scope and Extensions

The evaluation scope was intentionally centred on the evidence chain rather than on platform-scale benchmarking. The worked example uses a representative EDGAR fixture and a controlled run so that the links between source reference, release evidence, runtime manifest, quality result, lineage, and dataset version can be inspected directly. Production-scale throughput, long-term retention behaviour, and sustained failure recovery are separate operational concerns that build on the same evidence model.

The development scope also reflects an intentional split between design target and iteration environment. The architecture is designed for the target AWS environment, but Docker-based Airflow was used during development because MWAA provisioning and teardown were too slow for normal iteration. This allowed the evidence model to be exercised repeatedly while keeping the target architecture as the point of reference.

The policy scope is focused on demonstrating where governance decisions attach to the pipeline. The prototype includes the structure for governance checks and quality checks. A broader policy catalogue could extend the same pattern to access-control rules, separation of duties, data classification checks, retention rules, and incident-response evidence.

8.5 Future Work

Future work should first explore a less centralised governance architecture. The most direct extension would be to split the current GitHub-dependent control plane into separate services for source control, release signing, artifact storage, policy decisions, attestations, and evidence retention. This would allow the system to continue preserving audit evidence even when one provider is unavailable or degraded.

Future work should also explore escalating environments. The current work uses a development harness to evaluate the evidence model against the target architecture. A fuller environment pathway would define a sequence of controlled environments. A development environment would test DAG shape, schema validation, and manifest construction. A staging environment would test MWAA, S3, Terraform state, OpenMetadata, and OpenLineage integration using non-production data. A production-like environment would then test release promotion, policy enforcement, and audit reconstruction under stricter access control. This would separate fast evidence-model iteration from managed-service validation without changing the architecture being evaluated.

Further work should also test negative and degraded cases. The current example focuses on a successful governed run. A fuller evaluation should include failed quality gates, unresolved governance references, failed attestation, missing metadata publication, GitHub outage scenarios, and replay of historical dataset

versions. These cases would show whether the architecture only records success or whether it can also explain failure.

Finally, future work should increase the variety of data and workflow patterns. The EDGAR example is useful because it is structured and repeatable, but regulated organisations often operate mixed batch, streaming, API, and machine-learning pipelines. Applying the evidence-chain model to those cases would test whether the architecture generalises beyond a single ETL pattern.

Chapter 9

Conclusion

This thesis set out to answer the question:

> How can a DevSecOps toolchain be designed to support ETL workflows in regulated industries, enhancing auditability and data governance?

The answer developed in this thesis is that DevSecOps can support governed ETL when the delivery pipeline and the data pipeline are treated as one evidence system. In the proposed architecture, a dataset is not only the output of an Airflow run. It is the result of a governed change, an approved release, a specific infrastructure state, a known code bundle, a quality decision, and a recorded lineage path. Auditability is therefore designed into the workflow rather than reconstructed after the workflow has already completed.

9.1 Summary of Contributions

The first contribution of this thesis is the architecture itself. Chapter 5 presented a governed ETL architecture at escalating levels of detail, beginning with the high-level system structure and moving through process maps, data flows, metadata flows, and a worked execution path. This architecture extends a standard Airflow and AWS-style ETL pipeline with change governance, release manifests, policy checks, quality gates, lineage publication, metadata records, and audit-chain artifacts.

The second contribution is the implemented prototype. The implementation used a governed EDGAR pipeline to show how the architecture can be materialised in code. The prototype produced run manifests, release references, change references, code hashes, quality results, OpenLineage-style events, OpenMetadata-compatible records, and audit-chain files. Although the demonstration was intentionally scoped, it showed that a produced dataset version can be traced back to the authority, code, infrastructure, and controls that produced it.

The third contribution is the evaluation framework and requirement mapping. Chapter 3 translated auditability and governance concerns into five system requirements: governance accountability, information security assurance, operational risk control, continuous control monitoring, and end-to-end evidence chain. Chapter 7 then evaluated the prototype against these requirements and linked each one back to the auditability dimensions of normativity, accountability, referentiality, reliability, and verifiability.

9.2 Findings

The main finding is that auditability improves when evidence is captured as part of the delivery and runtime workflow. The baseline pipeline could produce data and operational logs, but it did not bind the output to a complete governance chain. The proposed architecture changed this by making the run manifest and metadata artifacts central to the system.

The worked example in Chapter 6 demonstrated this point. Starting from the example dataset version, the evidence chain could be followed to the audit-chain artifact, run manifest, release manifest, Terraform reference, change record, code hashes, quality decision, lineage event, and metadata records. This does not remove the need for human judgement in an audit, but it does change what the auditor is judging. Instead of assembling evidence from unrelated logs and repository history, the auditor can inspect a structured chain of fields and artifacts.

The evaluation also showed that the architecture is strongest when controls produce evidence, not only decisions. A policy check that blocks deployment is useful, but it becomes auditable when its result is retained and linked to the release or run. A data contract is useful, but it becomes auditable when the quality result records that the contract was checked before promotion. A lineage record is useful, but it becomes stronger when it points to the same dataset version and manifest as the rest of the evidence chain.

9.3 Scope and Reflection

The demonstration was intentionally scoped around the evidence model. The EDGAR pipeline and representative run were used to test whether a governed ETL system could connect release identity, runtime state, quality evidence, lineage, and metadata into a coherent audit chain. Production-scale reliability, multi-team access-control behaviour, and long-term retention are operational extensions of that same architecture rather than the focus of the evaluation.

The most important design trade-off is reliance on GitHub. GitHub is useful because it brings source control, issues, pull requests, CI/CD, and release automation close together. At the same time, that convenience concentrates trust. The outages encountered during development made this risk clear. A future production version of this architecture should split some of these responsibilities across

independent services so that audit evidence is not dependent on one platform remaining continuously available and internally consistent.

9.4 Closing Remarks

This thesis demonstrates that DevSecOps can enhance ETL governance when it is used to create durable evidence, not only automation. The proposed architecture does not treat compliance as a final review step. It treats governance references, release identity, code integrity, quality decisions, lineage, and metadata as normal outputs of the pipeline.

For regulated environments, this matters because trust in data depends on more than correct transformation logic. It depends on whether the organisation can explain how the data was produced, who authorised the change, which controls ran, and how the output can be reconstructed. The architecture developed in this thesis provides a practical path toward that goal by making auditability a designed property of the ETL system.

Appendix A

Implementation Version Details

This appendix records the implementation versions and platform references used by the prototype. The tables are separated from Chapter 6 because they support repeatability, but they are not themselves the main results of the evaluation.

Table A.1: Runtime and orchestration plane versions.

Component	Version or source	Role
Prototype package	`0.1.0`	Versioned package for jobs, manifests, metadata, and scripts.
Python runtime	`>=3.11`	Minimum runtime declared by the package.
Apache Airflow	`2.9.3-python3.11`	DAG runtime used to mirror the MWAA orchestration interface during development.
MWAA	`mw1.small`	Target managed Airflow environment class.
OpenMetadata server	`1.12.6`	Catalogue and lineage-facing metadata service.
OpenMetadata database image	`1.12.6`	Metadata database image used by the catalogue service.
Elasticsearch	`7.16.3`	Search backend used by OpenMetadata.

Table A.2: Governance and CI/CD plane versions.

Component	Version or source	Role
GitHub Actions Python	`3.12`	Runtime used by PR and release controls.
GitHub Actions checkout	`checkout@v4`	Checks out source for CI and release workflows.
GitHub Actions Python setup	`setup-python@v5`	Provides the workflow Python runtime.
GitHub Actions artifacts	`upload-artifact@v4`	Uploads control and runtime evidence.
GitHub attestation	`actions/attest@v4`	Generates release-bundle provenance.
OPA setup action	`setup-opa@v2`	Installs OPA for governance policy evaluation.
OPA version	`latest`	Evaluates pull-request change-gate policies.
Ruff	`0.11.10`	Python lint control in the PR workflow.
Pytest	`7.4.4`	Unit, integration, and ETL smoke validation.

Table A.3: Cloud target and infrastructure plane versions.

Component	Version or source	Role
Terraform	`>=1.6.0`	Infrastructure-as-code layer for AWS resources.
AWS provider	`~>6.39`	Terraform provider constraint for AWS.
S3 bucket versioning	`Enabled`	Versioning setting applied to raw, curated, manifest, and MWAA artifact buckets.
S3 server-side encryption	`AES256`	Default encryption setting applied to the storage buckets.
S3 public access block	`true` controls	Public ACL and policy access blocked for the storage buckets.

Table A.4: Evidence and release plane references.

Artifact	Version or reference	Role
OpenLineage schema	`1-0-0` schema URL	Shape used for emitted lineage events.
Governed change	`EDGAR-1`	Initial governed EDGAR implementation record.
Release manifest	`dev-edgar-v1`	Sample governed release for replay and tests.
Dataset version	`9b5fd26a2005c02b`	Promoted gold dataset version used for reconstruction.
Code artifact hashes	5 SHA-256 hashes	Hashes for the DAG and ingest, transform, quality, and promotion jobs.

Appendix B

Abbreviations

APRA	Australian Prudential Regulation Authority
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Delivery
CPG	Prudential Practice Guide (APRA)
CPS	Prudential Standard (APRA)
DAST	Dynamic Application Security Testing
DAG	Directed Acyclic Graph
DLC	Data Lifecycle
DSR	Design Science Research
ELT	Extract–Load–Transform
EMR	Elastic MapReduce
EC2	Elastic Compute Cloud
ETL	Extract–Transform–Load
GDPR	General Data Protection Regulation
GHCR	GitHub Container Registry
GitOps	Git-based Operations
HIPAA	Health Insurance Portability and Accountability Act
IaC	Infrastructure as Code
IAM	Identity and Access Management
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
ML	Machine Learning
MLOps	Machine Learning Operations
NIST	National Institute of Standards and Technology
PaC	Policy as Code
RBAC	Role-Based Access Control
S3	Simple Storage Service
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SBOM	Software Bill of Materials
SDLC	Software Development Lifecycle

SSDF	Secure Software Development Framework

Bibliography

[1] "ISO/IEC/IEEE International Standard – Systems and software engineering - Content of life-cycle information items (documentation)," Jul. 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8767110

[2] B. Achuthan and M. A. Alimohideen, "Shifting Gears: Integrating Security Audits into Automotive DevSecOps," in *2024 International Conference on Vehicular Technology and Transportation Systems (ICVTTS)*, vol. 1, Sep. 2024, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/10763937

[3] M. Anisetti, C. A. Ardagna, E. Damiani, and F. Gaudenzi, "A Semi-Automatic and Trustworthy Scheme for Continuous Cloud Service Certification," *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 30–43, Jan. 2020. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7831357/authors

[4] M. Anisetti, N. Bena, F. Berto, and G. Jeon, "A DevSecOps-based Assurance Process for Big Data Analytics," in *2022 IEEE International Conference on Web Services (ICWS)*, Jul. 2022, pp. 1–10. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9885738

[5] Australian Prudential Regulation Authority, "Prudential Practice Guide CPG 235: Managing Data Risk," Australian Prudential Regulation Authority (APRA), Canberra, ACT, Prudential Practice Guide CPG 235, Sep. 2013. [Online]. Available: https://www.apra.gov.au/sites/default/files/Prudential-Practice-Guide-CPG-235-Managing-Data-Risk_1.pdf

[6] ——, "Prudential Standard CPS 234: Information Security," Canberra, ACT, Jul. 2019. [Online]. Available: https://www.apra.gov.au/sites/default/files/cps_234_july_2019_for_public_release.pdf

[7] ——, "Prudential Standard CPS 510: Governance," Canberra, ACT, Jul. 2023. [Online]. Available: https://www.apra.gov.au/sites/default/files/prudential_standard_cps_510_governance.pdf

[8] ——, "Prudential Standard CPS 230: Operational Risk Management," Canberra, ACT, Jul. 2025. [Online]. Available: https:

//www.apra.gov.au/sites/default/files/2025-07/Prudential%20Standard%20CPS%20230%20Operational%20Risk%20Management%20-%20clean.pdf

[9] A. Ball, *Review of Data Management Lifecycle Models*. Bath, UK: University of Bath, Feb. 2012.

[10] C. Castellanos, C. A. Varela, and D. Correal, "ACCORDANT: A domain specific-model and DevOps approach for big data analytics architectures," *Journal of Systems and Software*, vol. 172, p. 110869, Feb. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121220302594

[11] T. Chen and H. Suo, "Design and Practice of Security Architecture via DevSecOps Technology," in *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*, Oct. 2022, pp. 310–313, iSSN: 2327-0594. [Online]. Available: https://ieeexplore.ieee.org/document/9930212

[12] D. B. Cruz, J. R. Almeida, and J. L. Oliveira, "Open Source Solutions for Vulnerability Assessment: A Comparative Analysis," *IEEE Access*, vol. 11, pp. 100 234–100 255, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10251527

[13] P. Cupoli, S. Earley, and D. Henderson, "DAMA-DMBOK2 Framework."

[14] S. Gregor and A. R. Hevner, "Positioning and Presenting Design Science Research for Maximum Impact," *MIS Quarterly*, vol. 37, no. 2, pp. 337–355, 2013, publisher: Management Information Systems Research Center, University of Minnesota. [Online]. Available: https://www.jstor.org/stable/43825912

[15] S. Gupta, M. Bhatia, M. Memoria, and P. Manani, "Prevalence of GitOps, DevOps in Fast CI/CD Cycles," in *2022 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COM-IT-CON)*, vol. 1, May 2022, pp. 589–596. [Online]. Available: https://ieeexplore.ieee.org/document/9850786

[16] R. Hernández, B. Moros, and J. Nicolás, "Requirements management in DevOps environments: a multivocal mapping study," *Requirements Engineering*, vol. 28, no. 3, pp. 317–346, Sep. 2023. [Online]. Available: https://doi.org/10.1007/s00766-023-00396-w

[17] A. Hevner, A. R, S. March, S. T, Park, J. Park, Ram, and Sudha, "Design Science in Information Systems Research," *Management Information Systems Quarterly*, vol. 28, pp. 75–, Mar. 2004.

[18] R. Kimball and J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004, iSBN: 9780764567575. [Online]. Available: https://www.oreilly.com/library/view/the-data-warehouse/9780764567575/

[19] B. O. Kose, "Agility Meets Compliance: The Convergence of Data Governance and DevSecOps," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 131–154. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch007

[20] W. C. Lenhardt, S. Ahalt, B. Blanton, L. Christopherson, and R. Idaszak, "Data Management Lifecycle and Software Lifecycle Management in the Context of Conducting Science | Journal of Open Research Software," Feb. 2014. [Online]. Available: https://openresearchsoftware.metajnl.com/articles/10.5334/jors.ax

[21] J. Li and H. Li, "Evolution of Application Security based on OWASP Top 10 and CWE/SANS Top 25 with Predictions for the 2025 OWASP Top 10," in *2025 International Conference on Inventive Computation Technologies (ICICT)*, Apr. 2025, pp. 1178–1183, iSSN: 2767-7788. [Online]. Available: https://ieeexplore.ieee.org/document/11004742

[22] D. Mahmud and M. Z. Ikbal, "THE ROLE OF ETL (EXTRACT-TRANSFORM-LOAD) PIPELINES IN SCALABLE BUSINESS INTELLIGENCE: A COMPARATIVE STUDY OF DATA INTEGRATION TOOLS," *ASRC Procedia: Global Perspectives in Science and Scholarship*, vol. 2, no. 1, pp. 89–121, Apr. 2022. [Online]. Available: https://global.asrcconference.com/index.php/asrc/article/view/29

[23] A. Mittal and V. Venkatesan, "Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation," in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Jul. 2025, pp. 197–203, iSSN: 2642-6587. [Online]. Available: https://ieeexplore.ieee.org/document/11126149

[24] H. A. Mohna, T. Barua, M. Mohiuddin, and M. M. Rahman, "AI-READY DATA ENGINEERING PIPELINES: A REVIEW OF MEDALLION ARCHITECTURE AND CLOUD-BASED INTEGRATION MODELS," *American Journal of Scholarly Research and Innovation*, vol. 1, no. 01, pp. 319–350, Apr. 2022. [Online]. Available: https://researchinnovationjournal.com/index.php/AJSRI/article/view/51

[25] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, "From Ad-Hoc Data Analytics to DataOps," in *Proceedings of the International Conference on Software and System Processes*, ser. ICSSP '20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 165–174, event-place: Seoul, Republic of Korea. [Online]. Available: https://doi.org/10.1145/3379177.3388909

[26] H. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in *Software Process Improvement and Capability Determination*, A. Mas, A. Mesquida, R. V. O'Connor, T. Rout, and A. Dorling, Eds. Cham: Springer International Publishing, 2017, pp. 17–29.

[27] S. Nai, A. Rifai, and A. Sadiq, "Data Governance, Key Insights, Strategic Challenges, and Future Imperatives:," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 1–16. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch001

[28] P. Pandey and A. Patel, "Integrating Security in Cloud-Native Development: A DevSecOps Approach to Resilient Software Systems," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 169–196. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch009

[29] A. Patel and P. Pandey, "Safeguarding Sensitive Data in the DevSecOps Pipelines: Strategies, Tools, and Best Practices for Modern Software Development," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 215–240. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch011

[30] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, publisher: Taylor & Francis, Ltd. [Online]. Available: https://www.jstor.org/stable/40398896

[31] A. Pogiatzis and G. Samakovitis, "An Event-Driven Serverless ETL Pipeline on AWS," *Applied Sciences*, vol. 11, no. 1, p. 191, Jan. 2021, publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2076-3417/11/1/191

[32] T. Rangnau, R. v. Buijtenen, F. Fransen, and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing

Tools in CI/CD Pipelines," in *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*, Oct. 2020, pp. 145–154, iSSN: 2325-6362. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9233212

[33] M. Souppaya, K. Scarfone, and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-218, Feb. 2022. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/218/final

[34] K. Soussane, B. E. Elbaghazaoui, and M. Amnai, "Integrating Security in DataOps: A Framework for Securing Data Pipelines in Real-time Analytics," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 197–214. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch010

[35] H. V. Sreepathy, B. Dinesh Rao, M. Kumar Jaysubramanian, and B. Deepak Rao, "Data Ingestions as a Service (DIaaS): A Unified Interface for Heterogeneous Data Ingestion, Transformation, and Metadata Management for Data Lake," *IEEE Access*, vol. 12, pp. 156 131–156 145, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10716380/

[36] R. Vayyala, "Data Lineage: Tracing Data Across Systems," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 73–98. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch004

[37] ——, "Ensuring Data Quality and Integrity in Modern Enterprises:," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 17–46. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch002

[38] ——, "Metadata Management and Its Role in Data Governance:," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 47–72. [Online]. Available: https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch003

[39] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, "Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 22, no. 5, pp. 847–859, May 2011. [Online]. Available: https://ieeexplore.ieee.org/document/5611497

[40] H. Weigand and P. Elsas, "Auditability as a Design Problem," in *2019 IEEE 21st Conference on Business Informatics (CBI)*, vol. 01, Jul. 2019, pp. 276–285, iSSN: 2378-1971. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8808092

[41] J. M. Wing, "The Data Life Cycle," *Harvard Data Science Review*, vol. 1, no. 1, Jul. 2019, publisher: The MIT Press. [Online]. Available: https://hdsr.mitpress.mit.edu/pub/577rq08d/release/4

[42] W. Yang, R. Fu, M. B. Amin, and B. Kang, "The Impact of Modern AI in Metadata Management," *Human-Centric Intelligent Systems*, vol. 5, no. 3, pp. 323–350, Sep. 2025. [Online]. Available: https://doi.org/10.1007/s44230-025-00106-5

[43] J. Zhang, J. Symons, P. Agapow, J. T. Teo, C. A. Paxton, J. Abdi, H. Mattie, C. Davie, A. Z. Torres, A. Folarin, H. Sood, L. A. Celi, J. Halamka, S. Eapen, and S. Budhdeo, "Best practices in the real-world data life cycle," *PLOS Digital Health*, vol. 1, no. 1, p. e0000003, Jan. 2022, publisher: Public Library of Science. [Online]. Available: https://journals.plos.org/digitalhealth/article?id=10.1371/journal.pdig.0000003

[44] Y. Zhou, Y. Yu, and B. Ding, "Towards MLOps: A Case Study of ML Pipeline Platform," in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 494–500. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9361315